

Introduction to Computer Vision



Lec.10: Recognition
2022.11.21

Today

Semantic segmentation

Object detection

Instance segmentation

Human pose estimation

Optical flow

Semantic segmentation

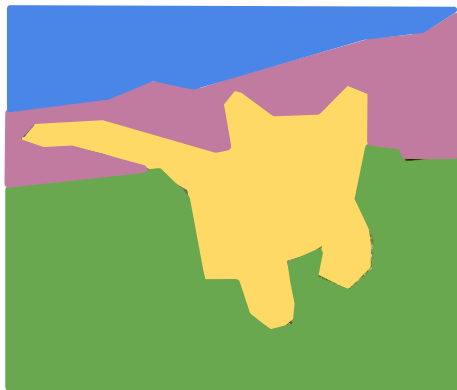
Classification



CAT

No spatial extent

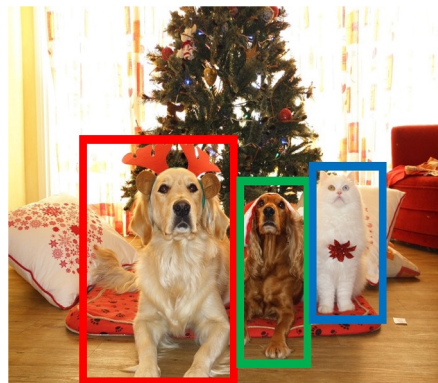
Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

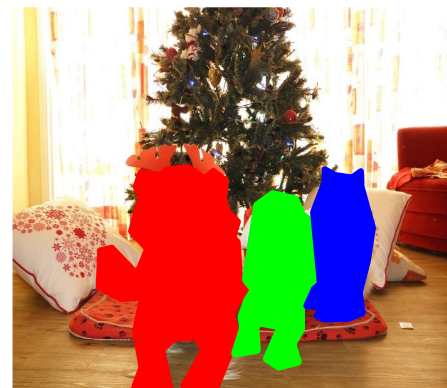
Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation



DOG, DOG, CAT

Semantic segmentation

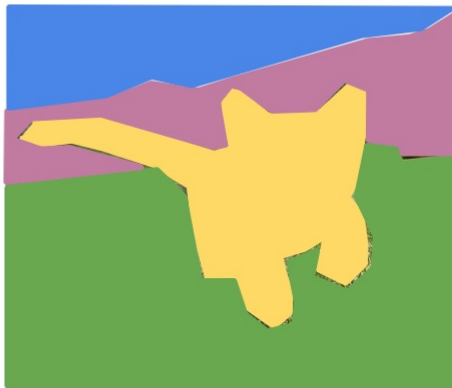
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation

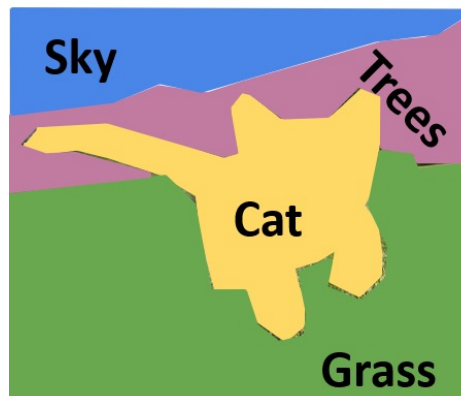


DOG, DOG, CAT

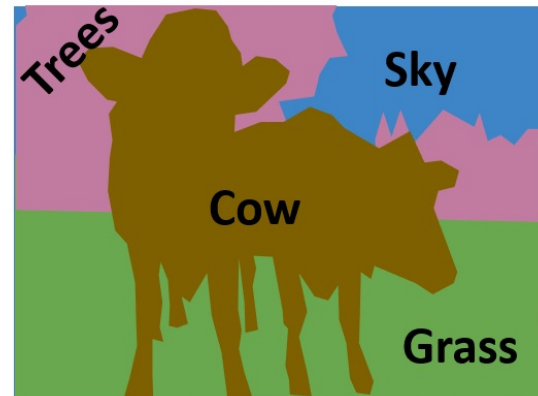
Semantic segmentation

Label each pixel in the image with a category label

Do not differentiate instances



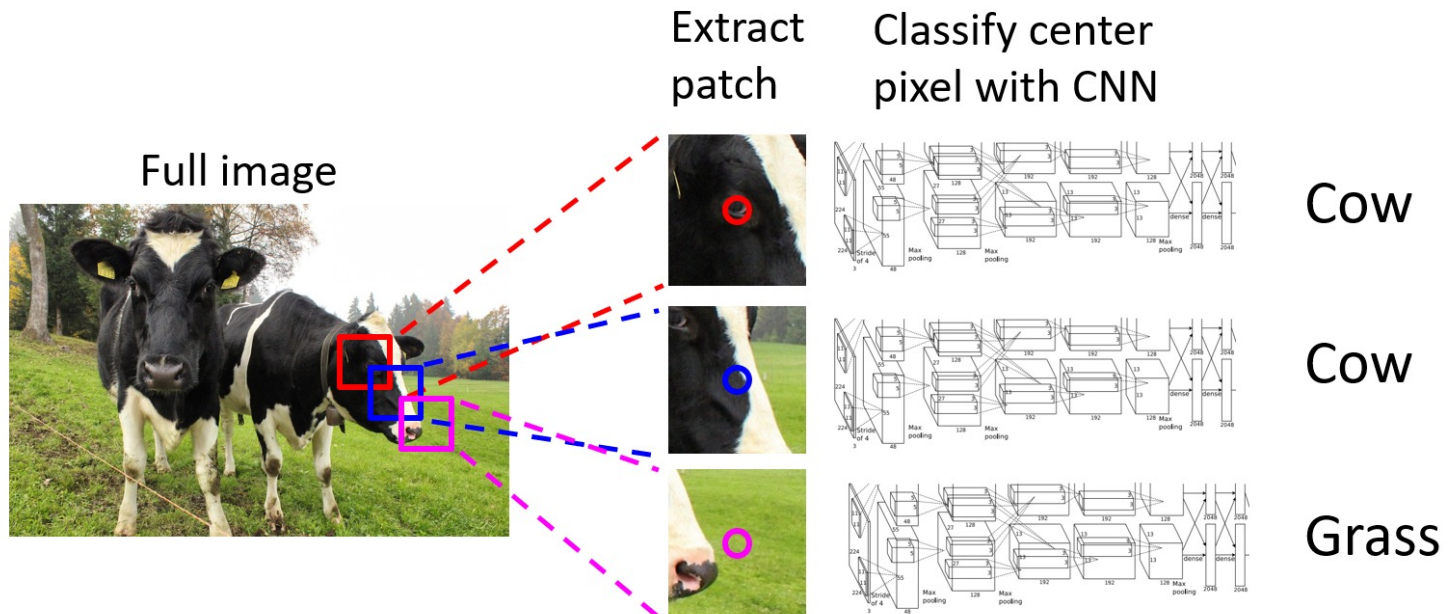
[This image is CC0 public domain](#)



Sliding window?

Very inefficient!

Limited receptive fields

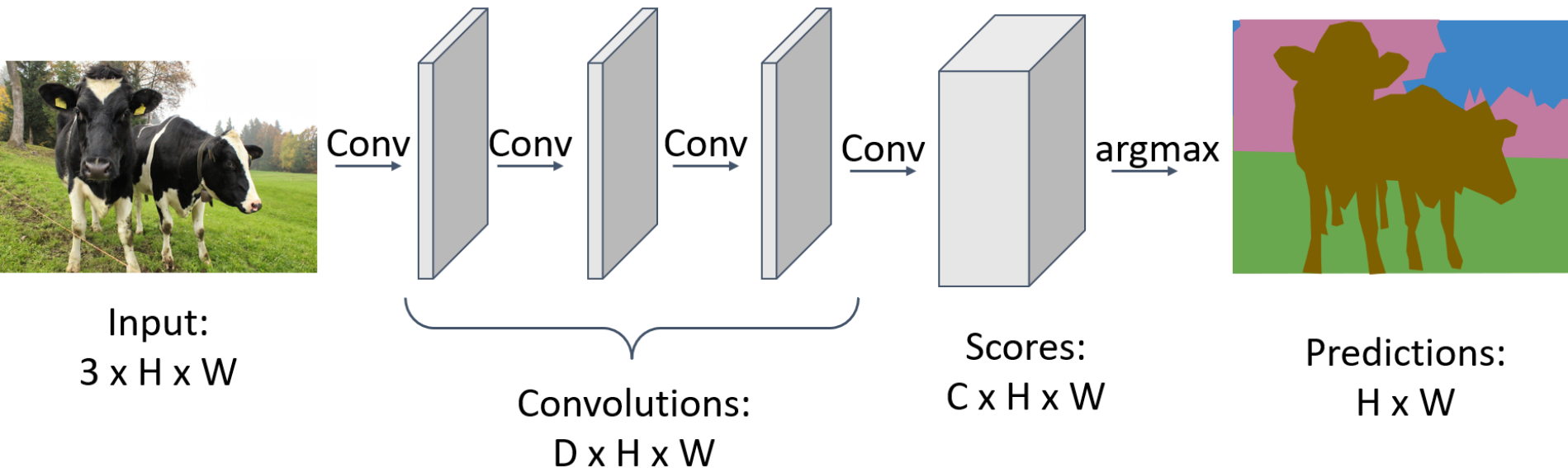


Fully convolutional network

Make predictions all at once

What is the loss function for training?

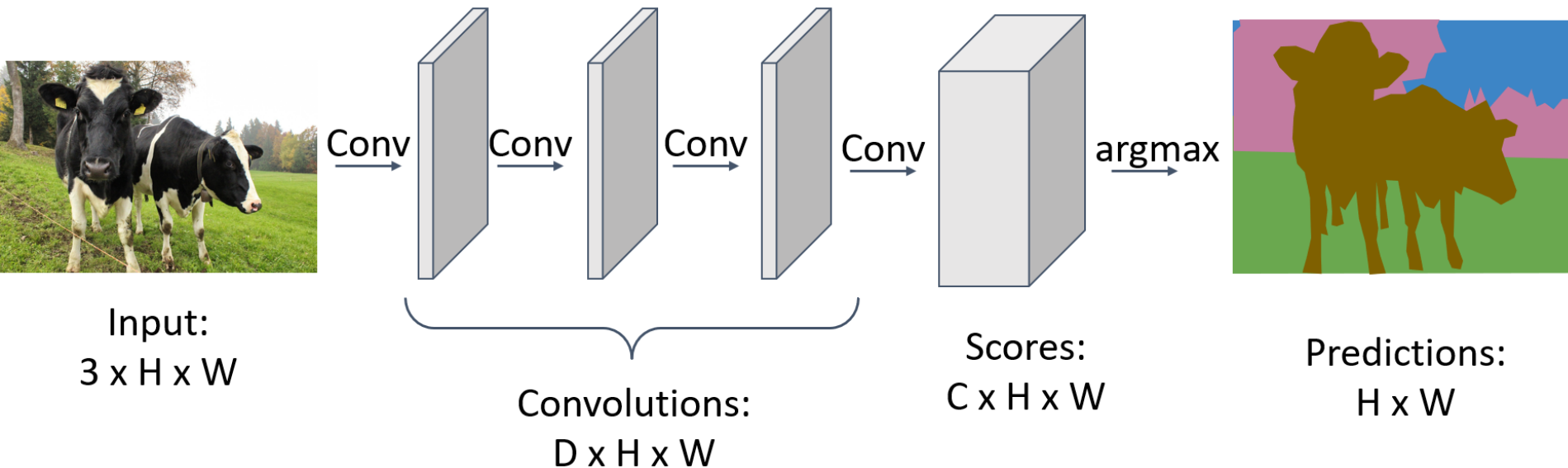
– Per-pixel cross-entropy



Fully convolutional network

Receptive field is linear in number of conv layers

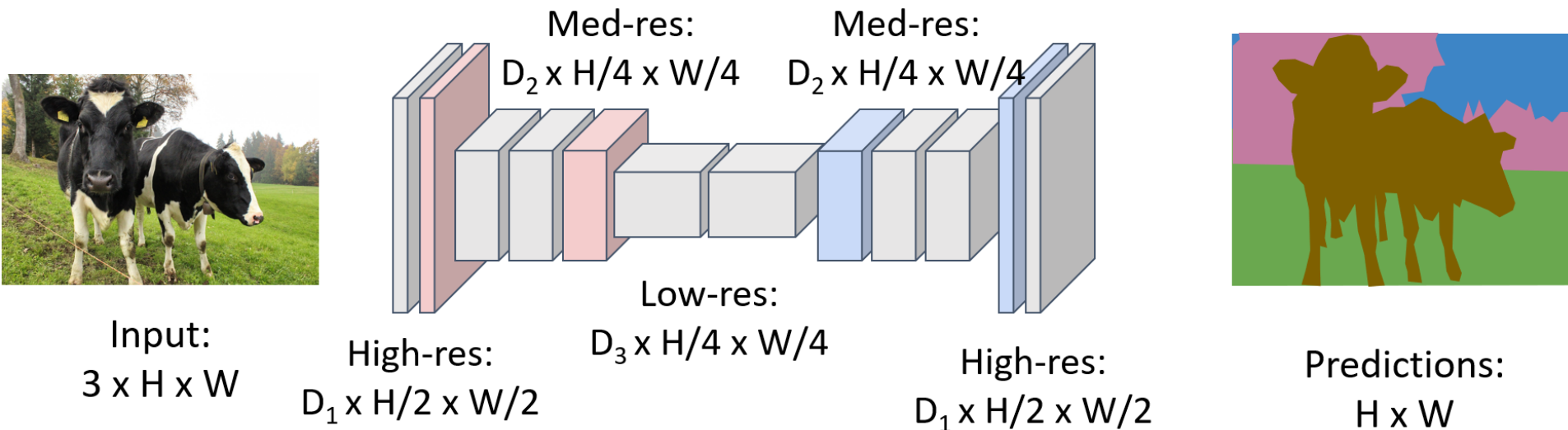
Convolution on high resolution is expensive



Fully convolutional network

How to improve efficiency and receptive field?

- Downsampling: pooling, strided conv
- How to maintain the resolution of output?



Unpooling

Bed of Nails

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

Nearest Neighbor

1	2
3	4



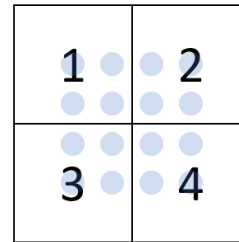
1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

Unpooling

Bilinear Interpolation

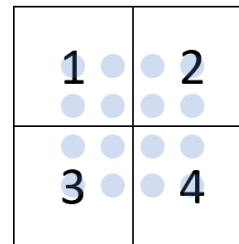


Input: $C \times 2 \times 2$

1.00	1.25	1.75	2.00
1.50	1.75	2.25	2.50
2.50	2.75	3.25	3.50
3.00	3.25	3.75	4.00

Output: $C \times 4 \times 4$

Bicubic Interpolation



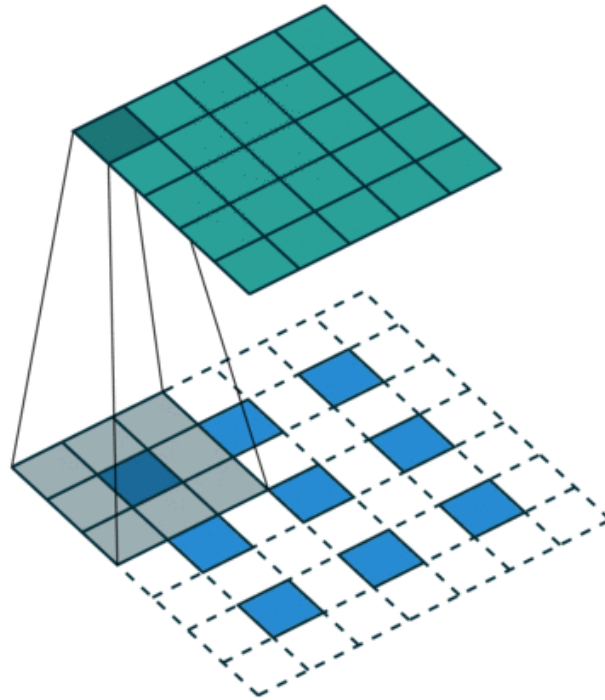
Input: $C \times 2 \times 2$

0.68	1.02	1.56	1.89
1.35	1.68	2.23	2.56
2.44	2.77	3.32	3.65
3.11	3.44	3.98	4.32

Output: $C \times 4 \times 4$

Transposed convolution

Unpooling followed by convolution



Fully convolutional network

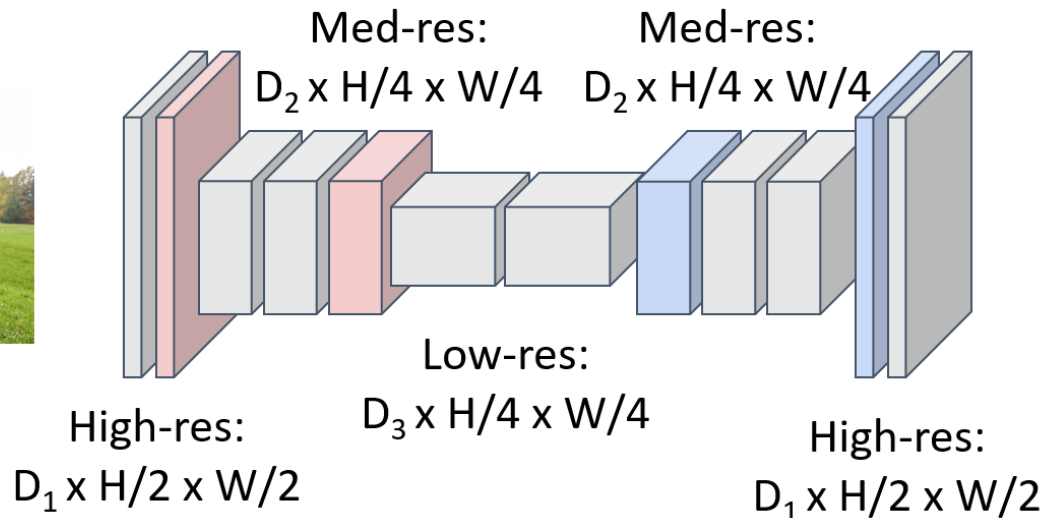
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling:
Interpolation,
transposed conv



Input:
 $3 \times H \times W$

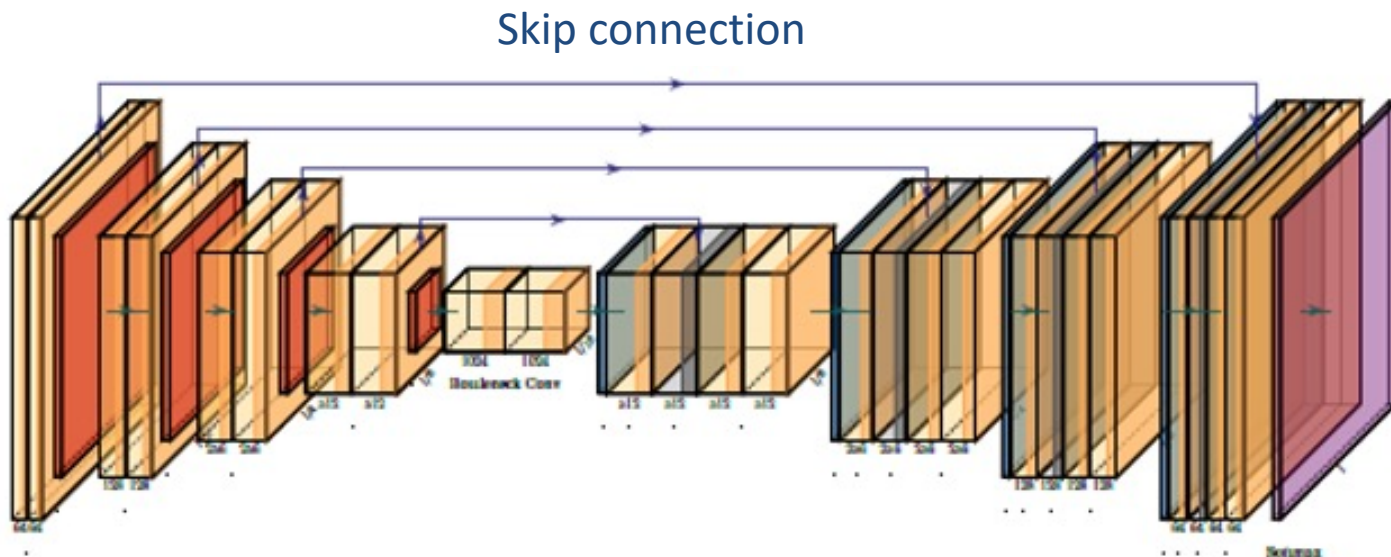


Predictions:
 $H \times W$

Loss function: Per-Pixel cross-entropy

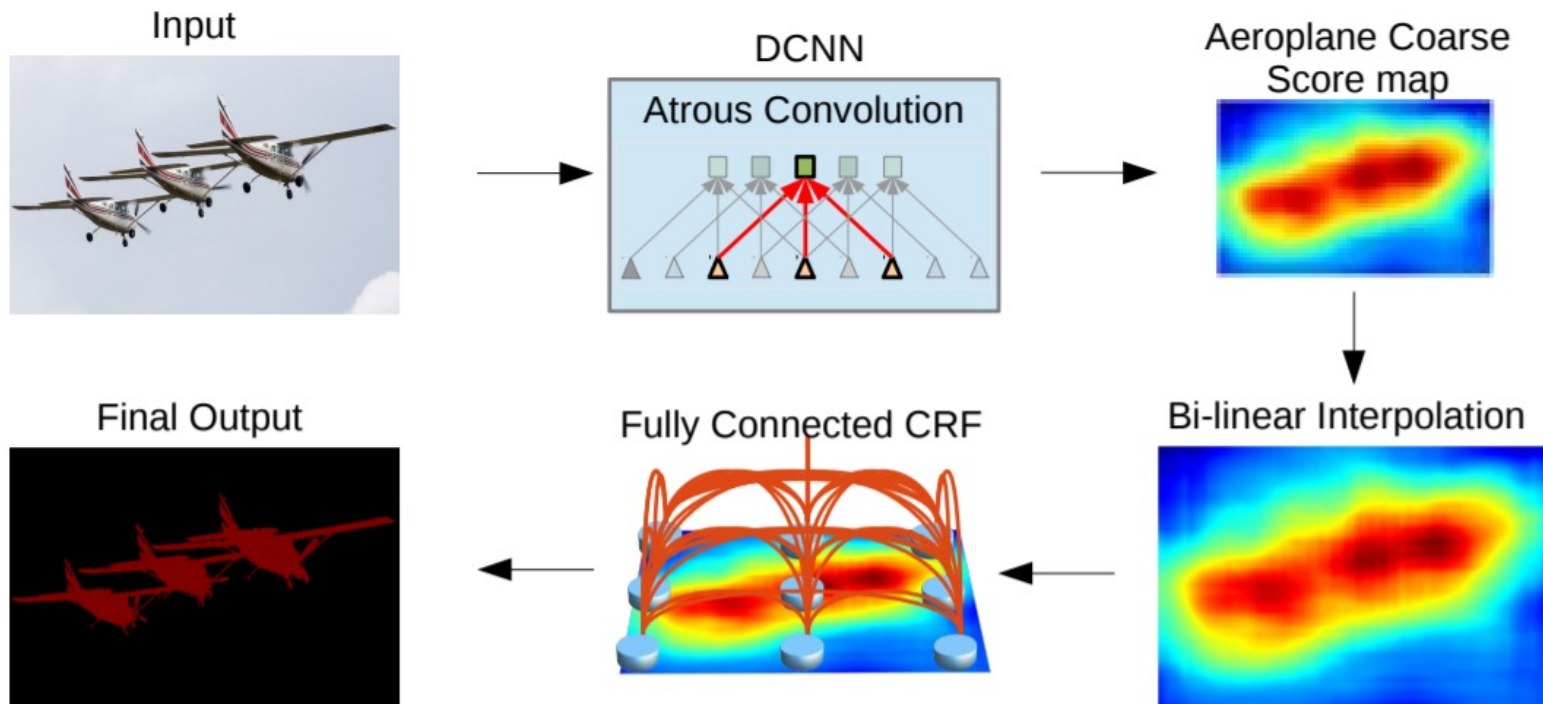
U-Net

Skip connection between downsampling and upsampling stages



DeepLab

FCN + CRF



Conditional random field

Energy function

$$E(\mathbf{x}) = \sum_i \theta_i(x_i) + \sum_{ij} \theta_{ij}(x_i, x_j)$$

Conditional random field

Unary potential

$$\theta_i(x_i) = -\log P(x_i)$$

The score map
given by network



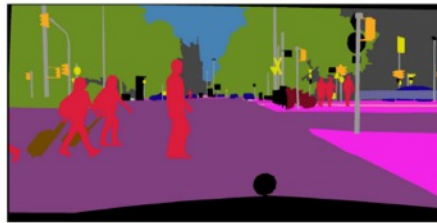
Pairwise potential

$$\theta_{ij}(x_i, x_j) = \mu(x_i, x_j) \left[w_1 \exp \left(-\frac{\|p_i - p_j\|^2}{2\sigma_\alpha^2} - \frac{\|I_i - I_j\|^2}{2\sigma_\beta^2} \right) + w_2 \exp \left(-\frac{\|p_i - p_j\|^2}{2\sigma_\gamma^2} \right) \right]$$

where $\mu(x_i, x_j) = 1$ if $x_i \neq x_j$, and zero otherwise

DeepLab

FCN + CRF



(a) Image

(b) G.T.

(c) Before CRF

(d) After CRF

Evaluation metric

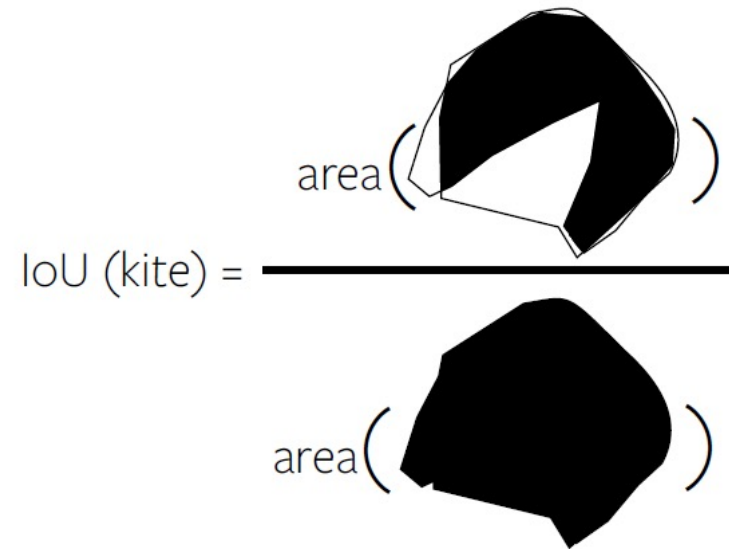
Per-pixel Intersection-over-union (IoU)



ground truth



prediction

$$\text{IoU (kite)} = \frac{\text{area}(\text{intersection})}{\text{area}(\text{union})}$$
The diagram illustrates the IoU formula for a kite. The numerator is 'area(intersection)' with a diagram of a kite shape where the intersection of two overlapping regions is highlighted. The denominator is 'area(union)' with a diagram of a kite shape where the union of two overlapping regions is highlighted.

Today

Semantic segmentation

Object detection

Instance segmentation

Human pose estimation

Optical flow

Object detection

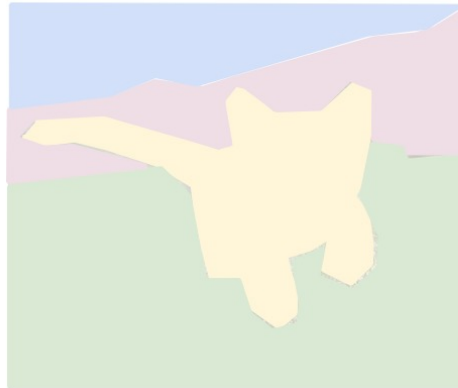
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

Instance Segmentation



DOG, DOG, CAT

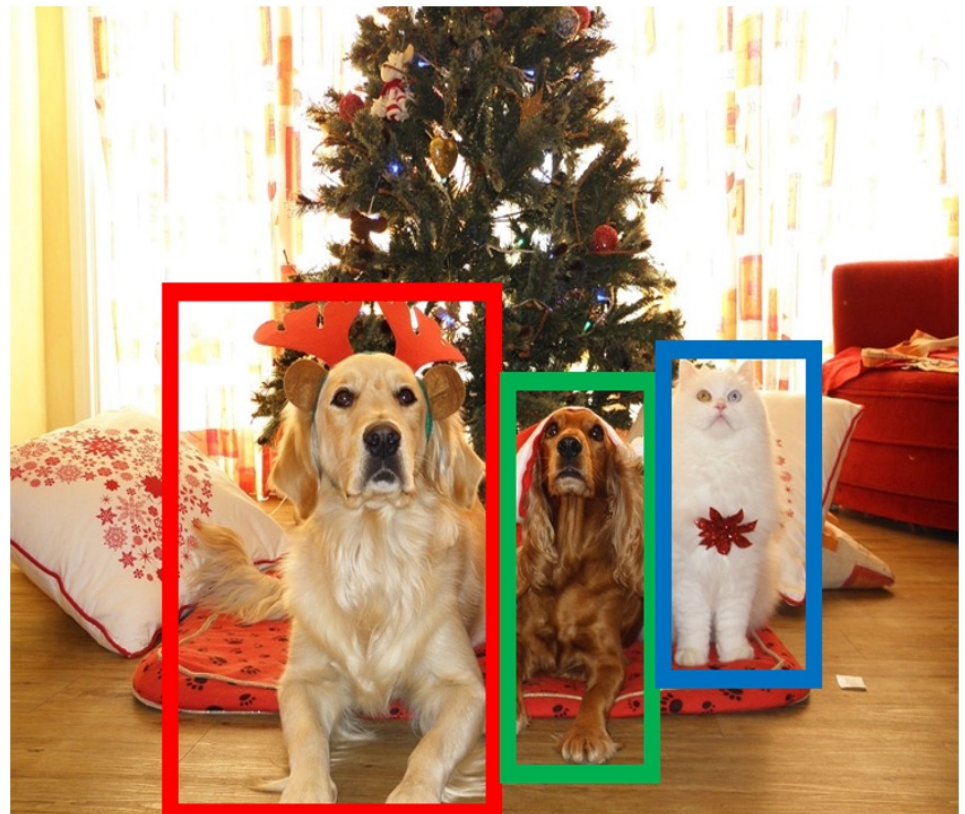
Object detection

Input: single RGB image

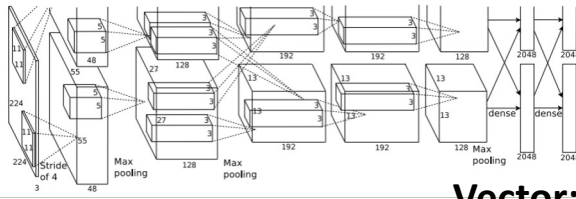
Output: A set of bounding boxes that denote objects

Bounding box (bbox)

- Class label
- Location (x,y)
- Size (w,h)



Detecting a single object



Vector:
4096

Fully
Connected:
4096 to 1000

Fully
Connected:
4096 to 4

“What”

Class Scores

Cat: 0.9

Dog: 0.05

Car: 0.01

...

Correct label:

Cat

**Softmax
Loss**

**Weighted
Sum Loss**

Box Coordinates

(x, y, w, h)

L2 Loss

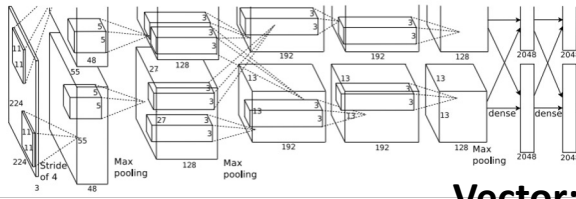
“Where”

Correct box:

(x', y', w', h')

Treat localization as a
regression problem!

Detecting a single object



Vector:
4096

Fully
Connected:
4096 to 1000

“What”

Class Scores

Cat: 0.9

Dog: 0.05

Car: 0.01

...

Correct label:

Cat

**Softmax
Loss**

**Weighted
Sum Loss**

Fully
Connected:
4096 to 4

Box Coordinates

(x, y, w, h)

L2 Loss

“Where”

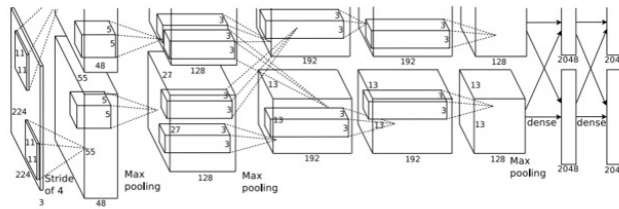
Correct box:

(x', y', w', h')

Problem: Images can have
more than one object!

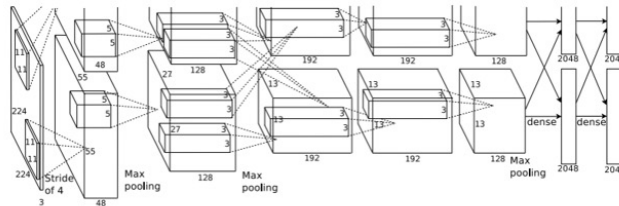
Detecting multiple objects

Need different numbers of output per image



CAT: (x, y, w, h)

4 numbers

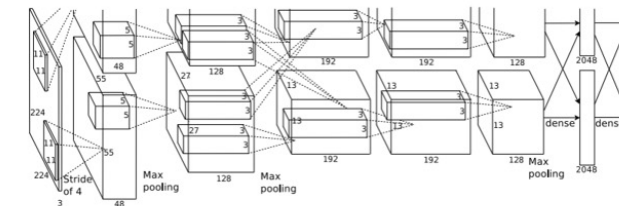


DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

16 numbers



DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

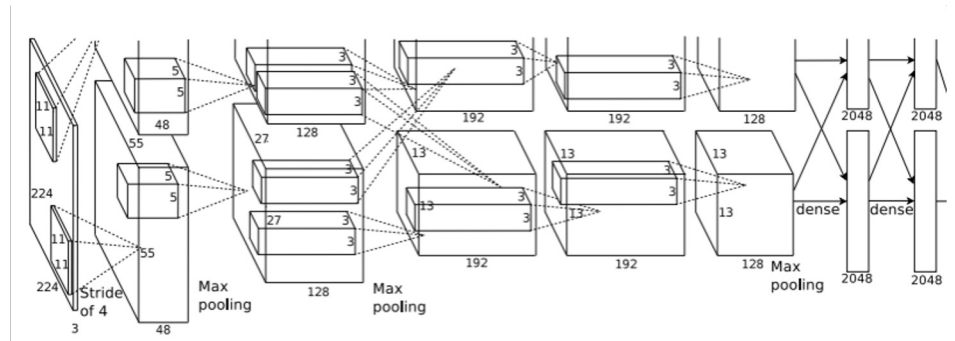
Many numbers!

....

Sliding window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

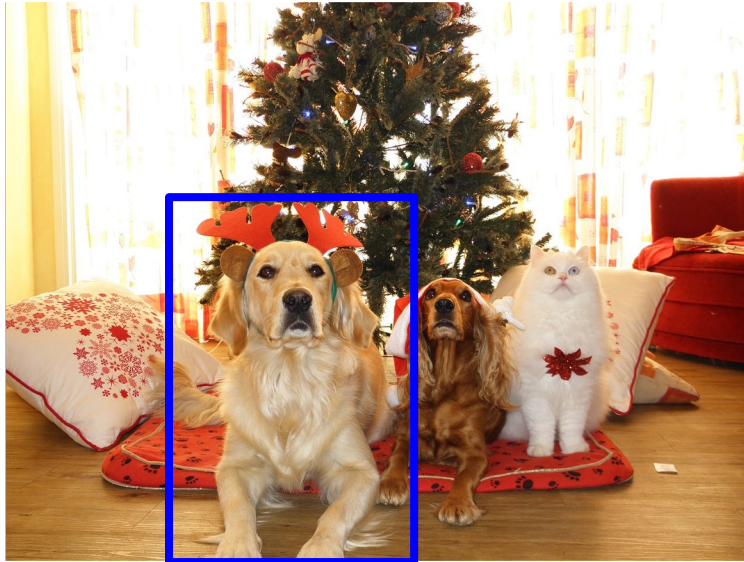


Dog? **NO**

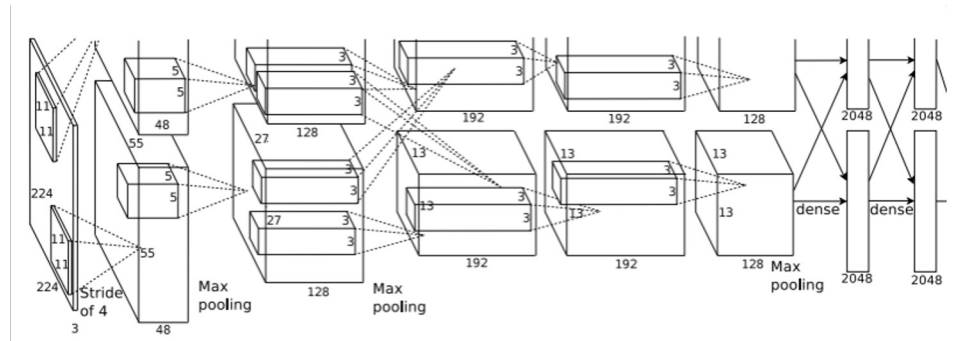
Cat? **NO**

Background? **YES**

Sliding window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? **YES**

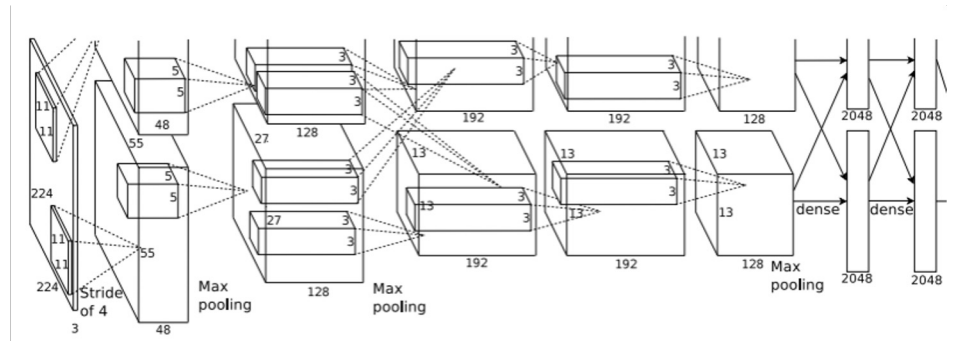
Cat? **NO**

Background? **NO**

Sliding window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? **YES**

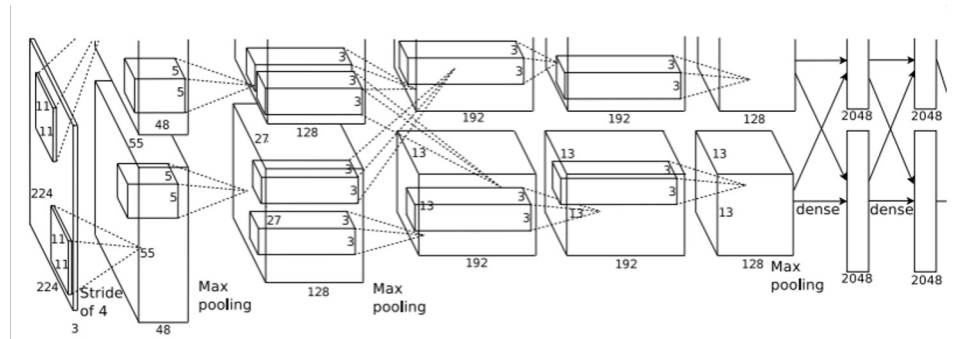
Cat? **NO**

Background? **NO**

Sliding window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? **NO**

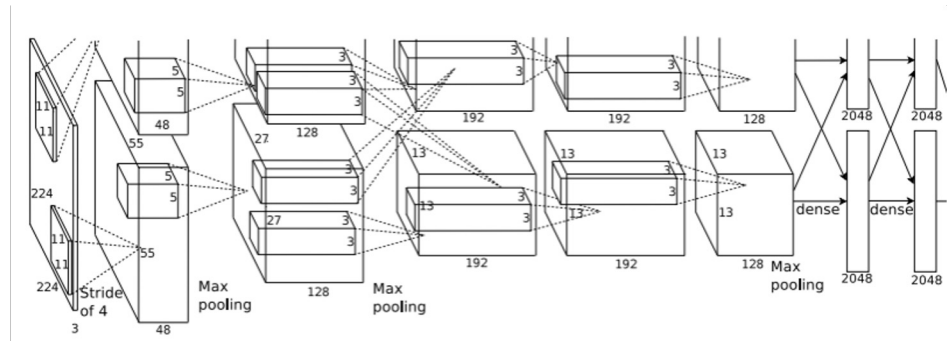
Cat? **YES**

Background? **NO**

Sliding window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Consider a box of size $h \times w$:

Possible x positions: $W - w + 1$

Possible y positions: $H - h + 1$

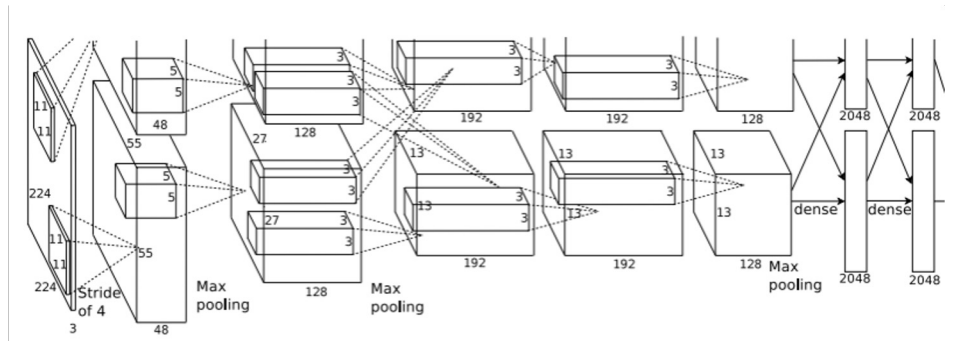
Possible positions:

$$(W - w + 1) * (H - h + 1)$$

Sliding window



Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



800 x 600 image has ~58M boxes!
No way we can evaluate them all

Region proposals

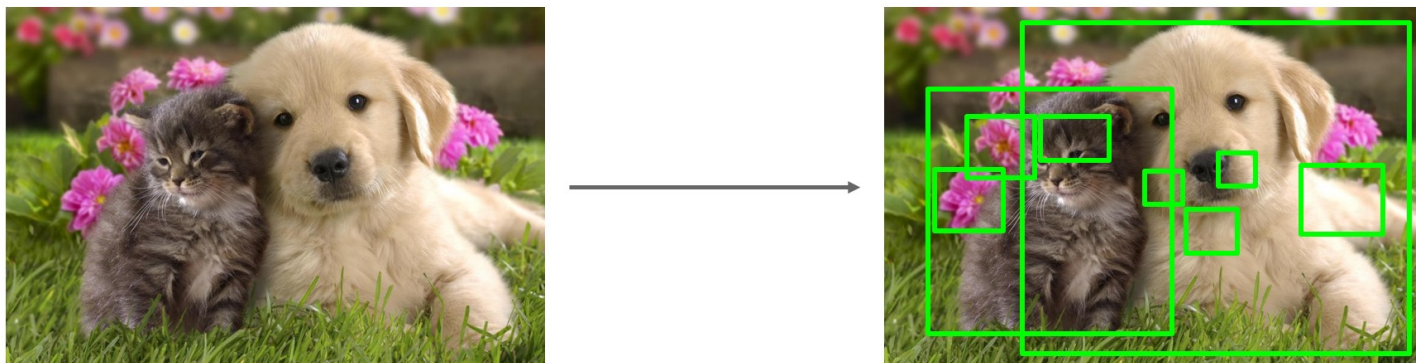
Find a small set of boxes that are likely to cover all objects

Often based on heuristics

- e.g. by over-segmentation

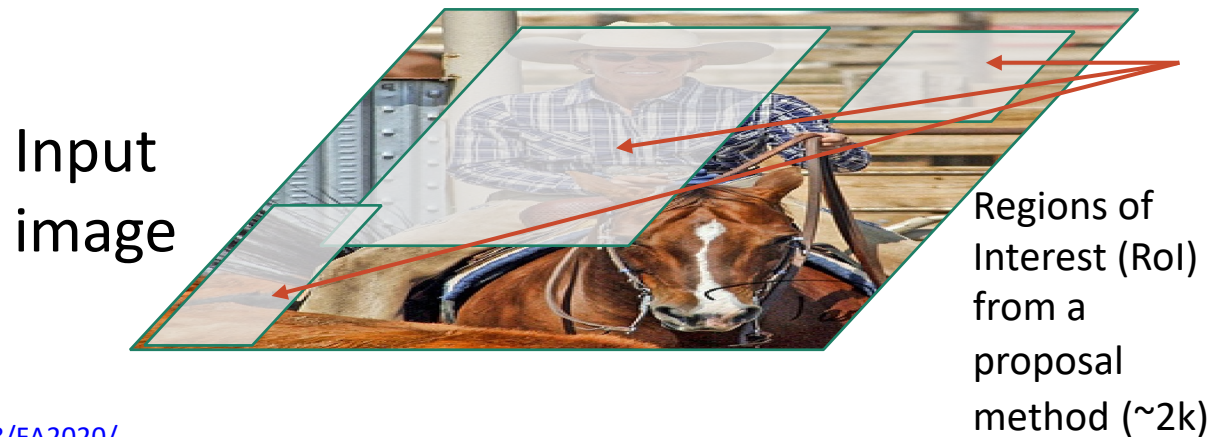
Relatively fast to run

- e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



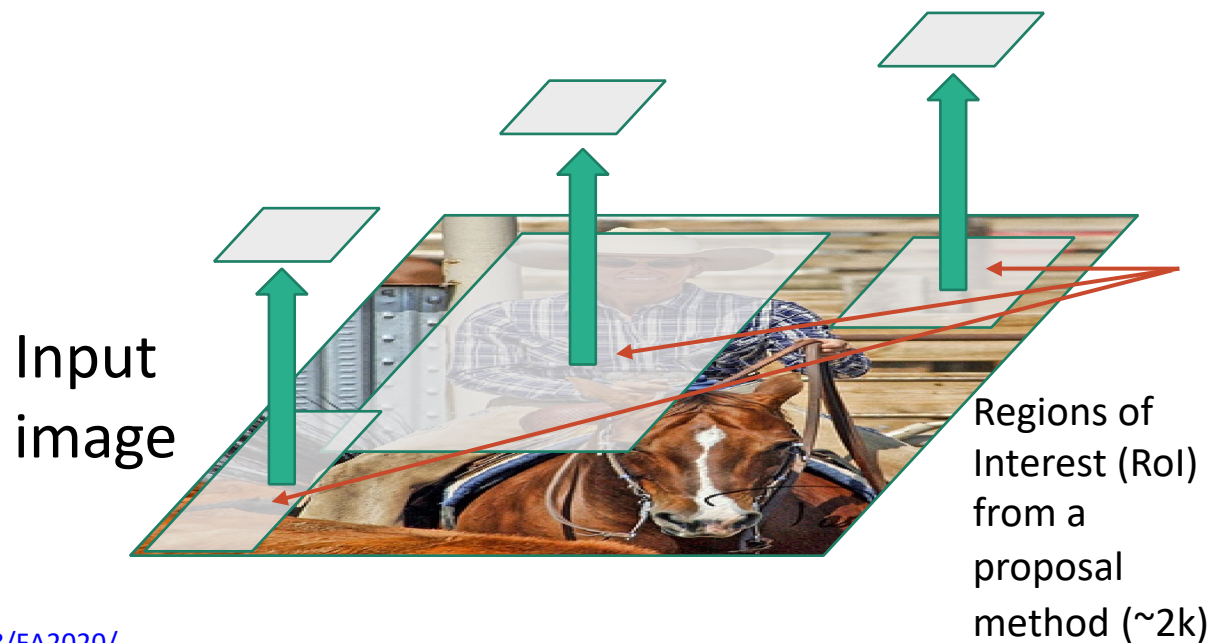
R-CNN

Run region
proposal method
to compute ~ 2000
region proposals



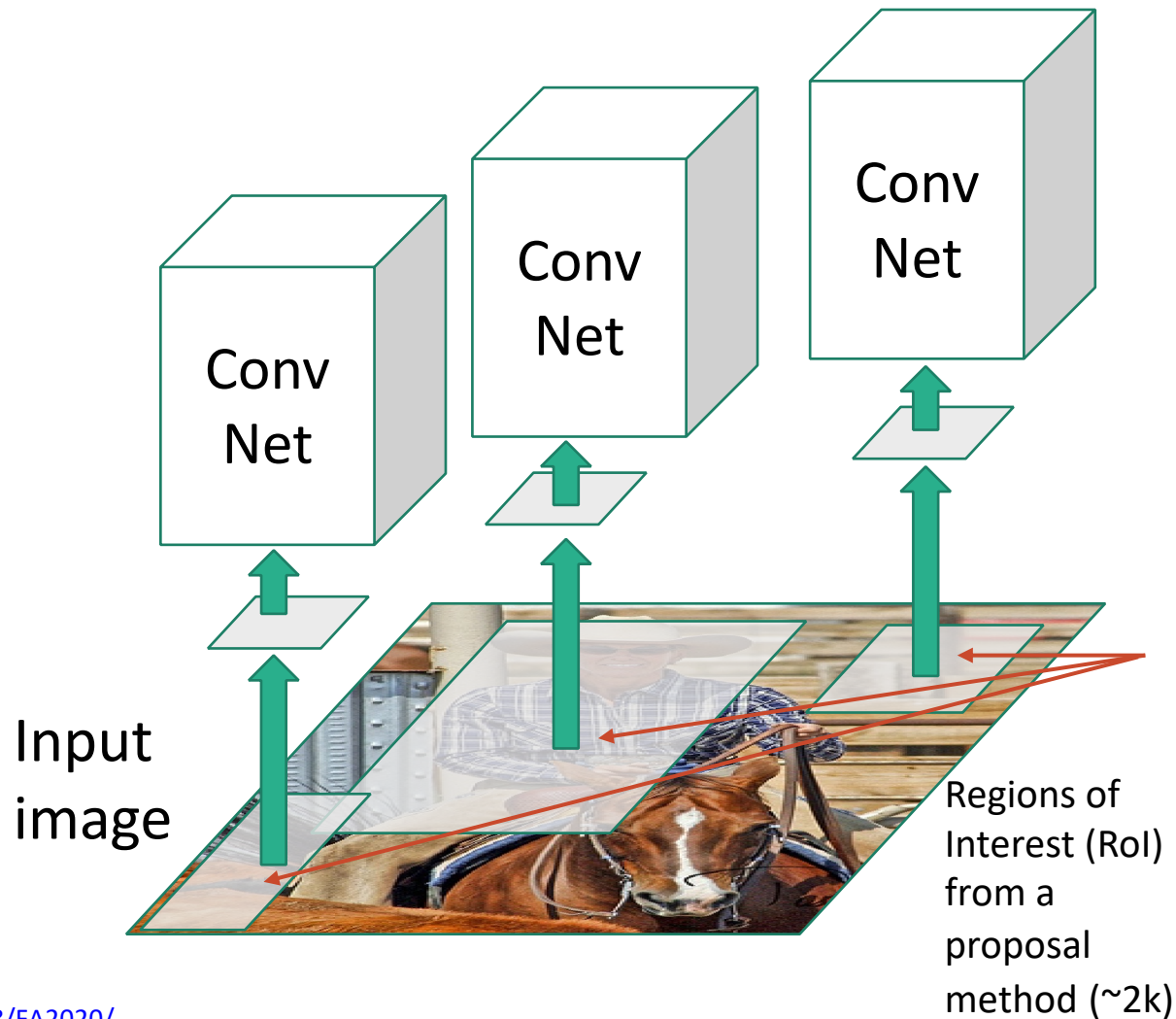
R-CNN

Resize each
region to 224x224
and run through
CNN



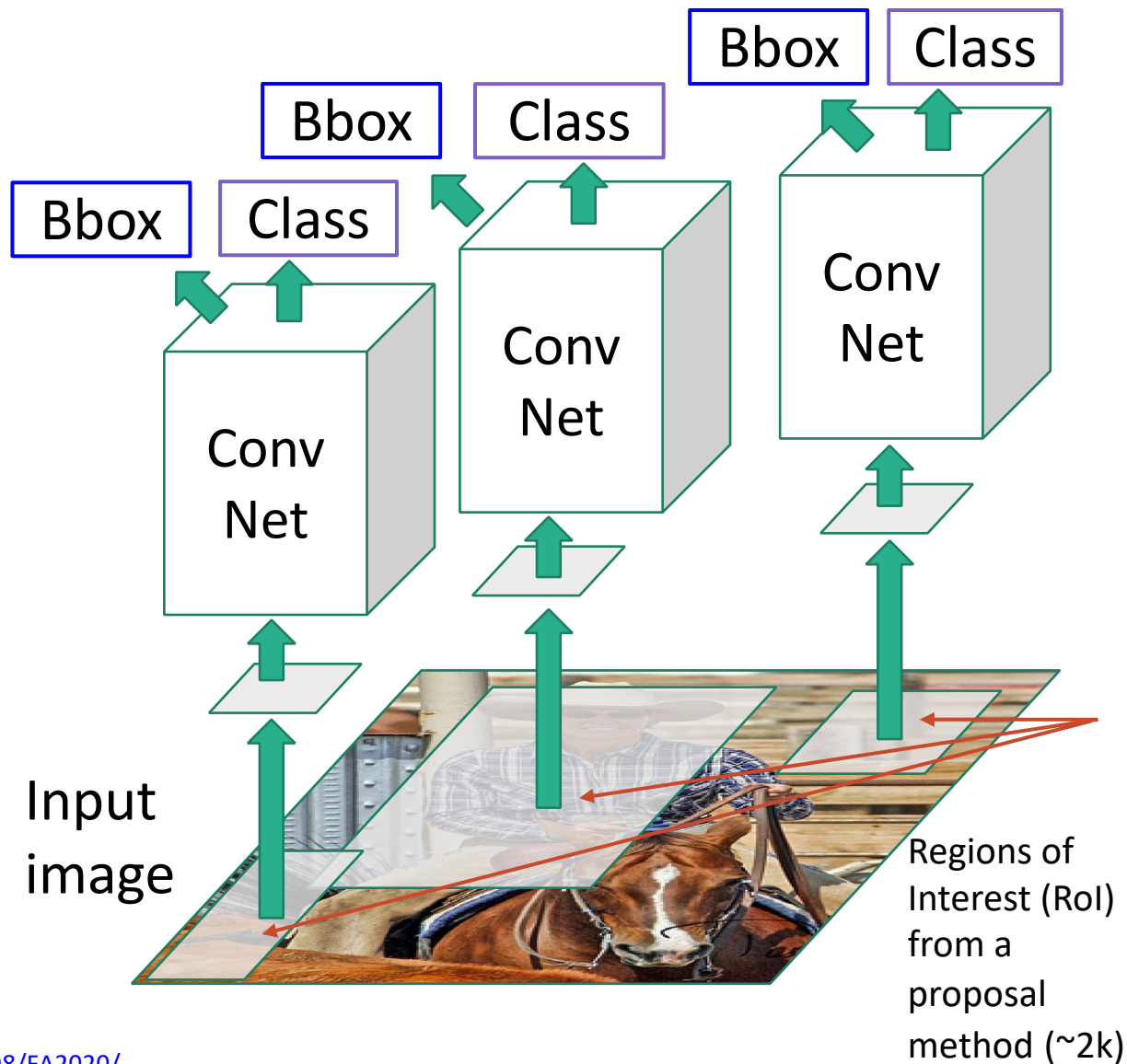
R-CNN

Resize each
region to 224x224
and run through
CNN



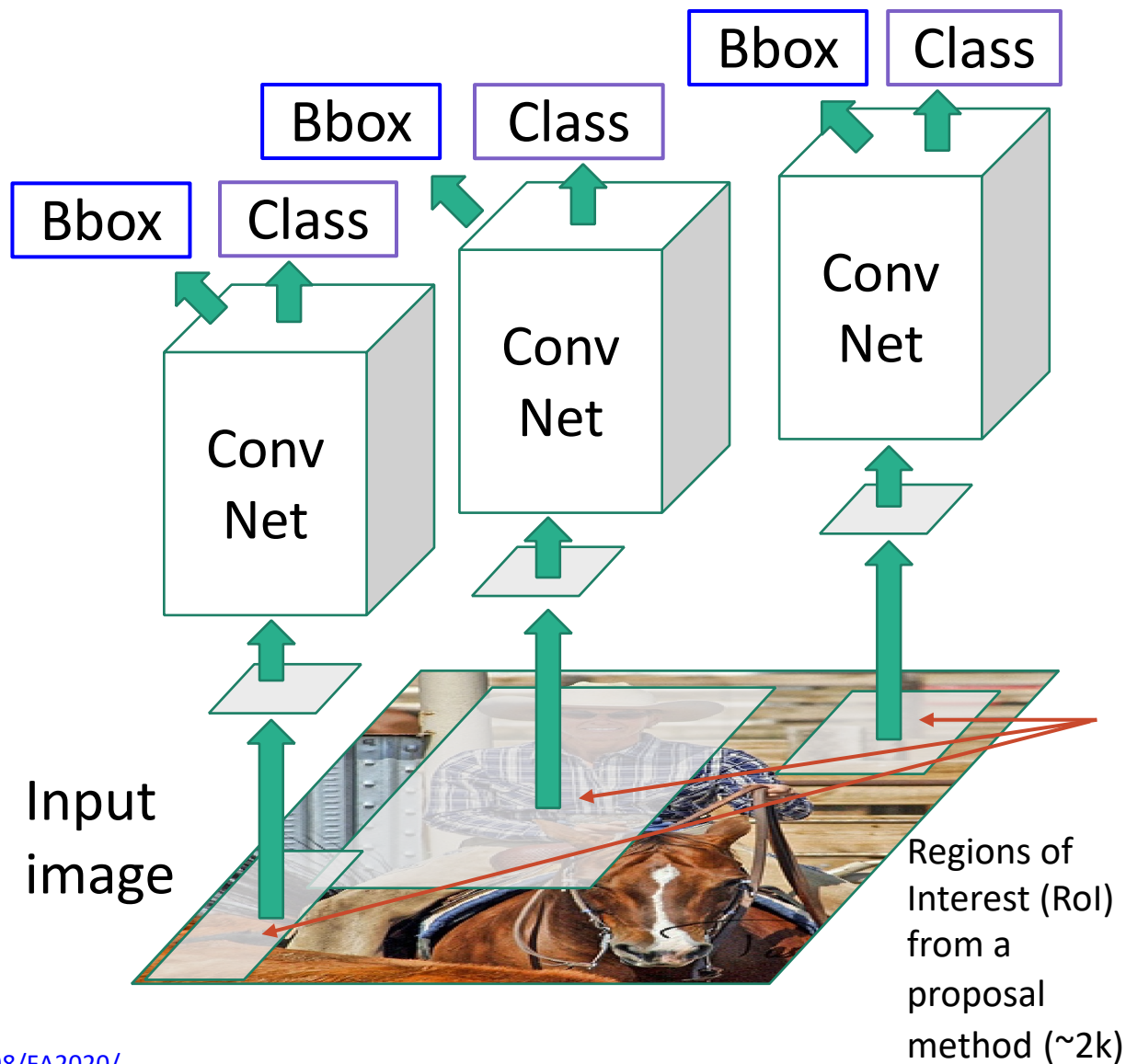
R-CNN

Predict class scores and bbox transform



R-CNN

Use scores to
select a subset of
region proposals
to output



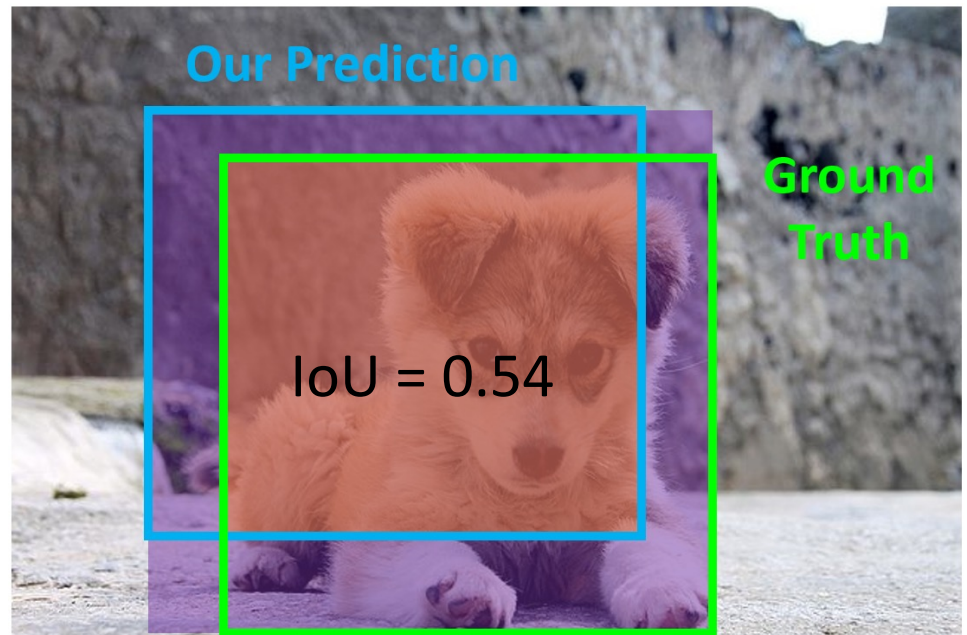
Evaluation metric

How can we compare our prediction with the ground-truth box?

Intersection over Union (IoU):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$\text{IoU} > 0.5$ is “decent”



Evaluation metric

How can we compare our prediction with the ground-truth box?

Intersection over Union (IoU):

$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$\text{IoU} > 0.5$ is “decent”

$\text{IoU} > 0.7$ is “pretty good”



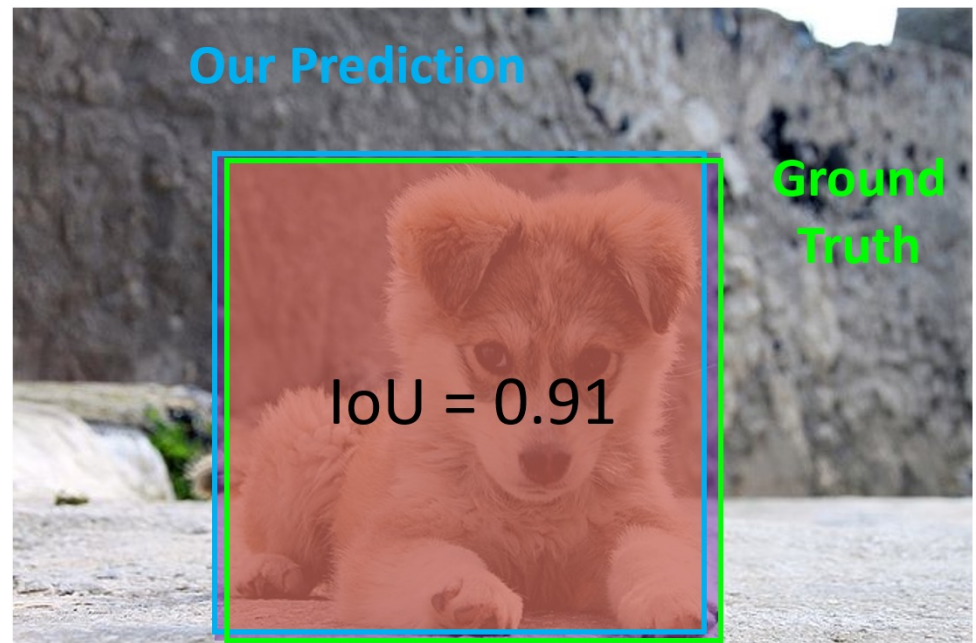
Evaluation metric

How can we compare our prediction with the ground-truth box?

Intersection over Union (IoU):

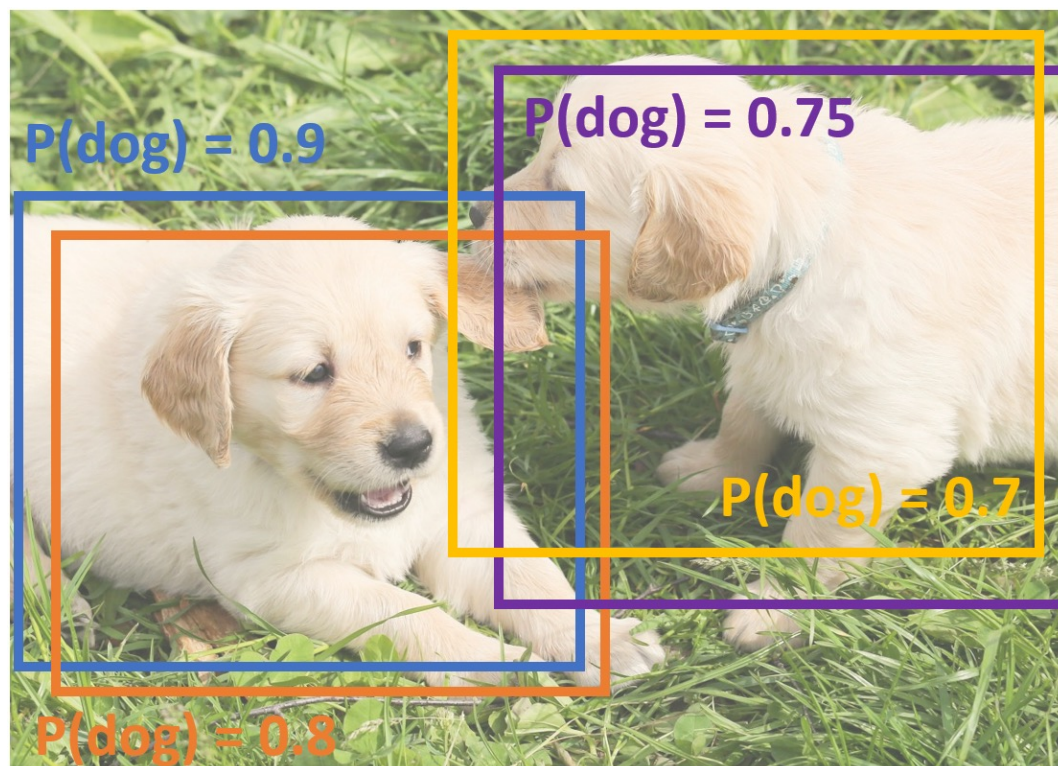
$$\frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$\text{IoU} > 0.5$ is “decent”
 $\text{IoU} > 0.7$ is “pretty good”
 $\text{IoU} > 0.9$ is “almost perfect”



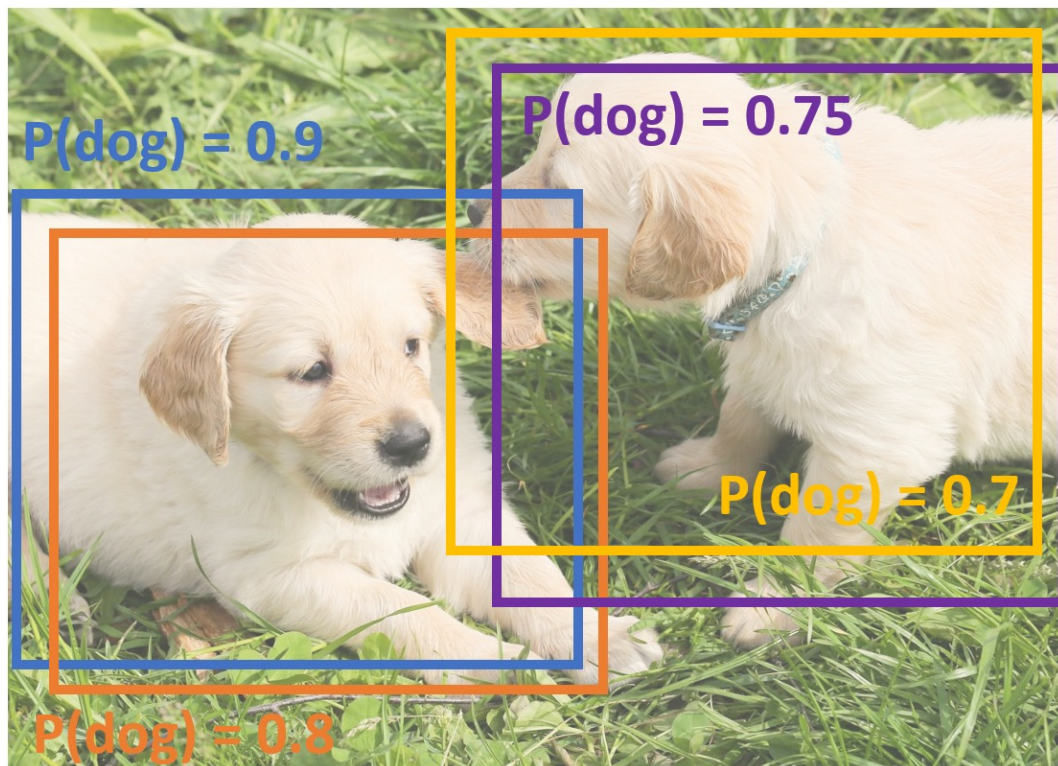
Non-Max Suppression

Object detectors often output many overlapping detections



Non-Max Suppression

1. Select the highest-scoring box
2. Eliminate lower-scoring boxes with $\text{IoU} > \text{threshold}$
3. If any boxes remain, goto 1



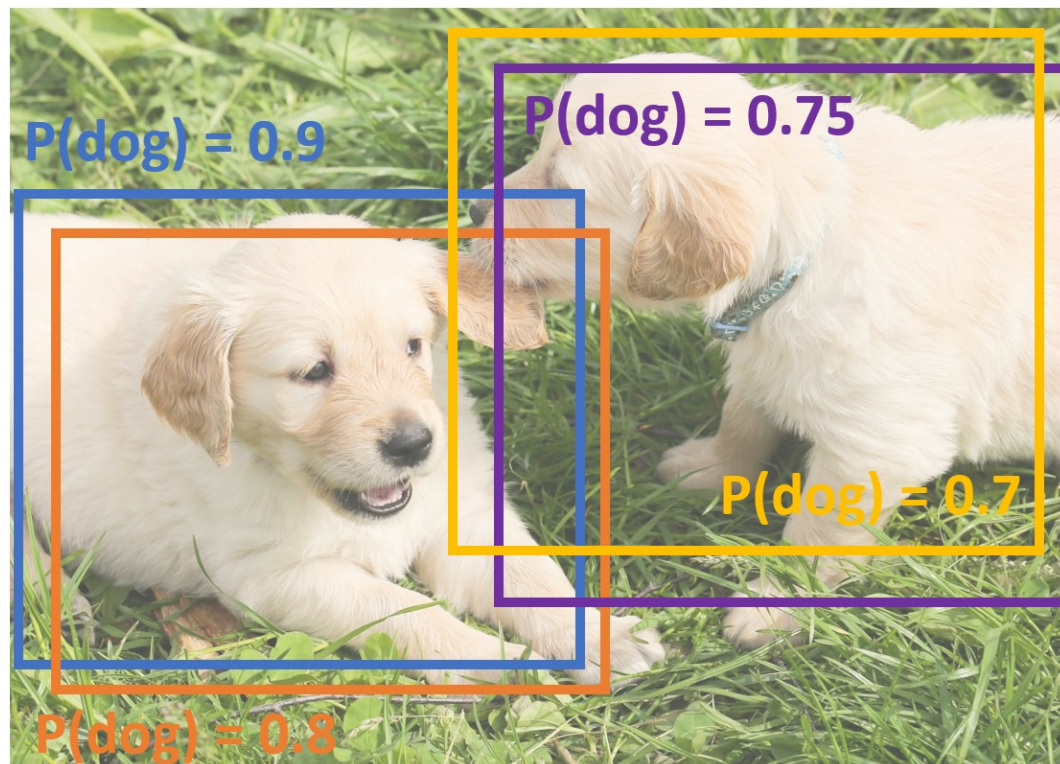
Non-Max Suppression

1. Select the highest-scoring box
2. Eliminate lower-scoring boxes with $\text{IoU} > \text{threshold}$
3. If any boxes remain, goto 1

$$\text{IoU}(\text{blue box}, \text{orange box}) = \mathbf{0.78}$$

$$\text{IoU}(\text{blue box}, \text{purple box}) = 0.05$$

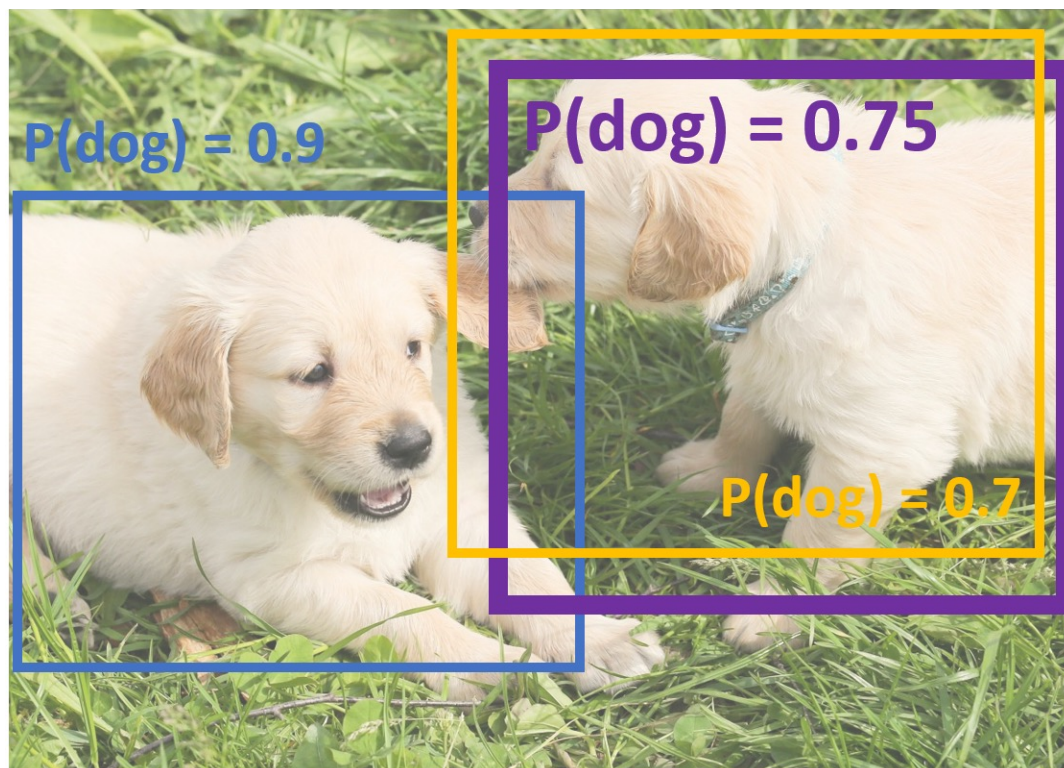
$$\text{IoU}(\text{blue box}, \text{yellow box}) = 0.07$$



Non-Max Suppression

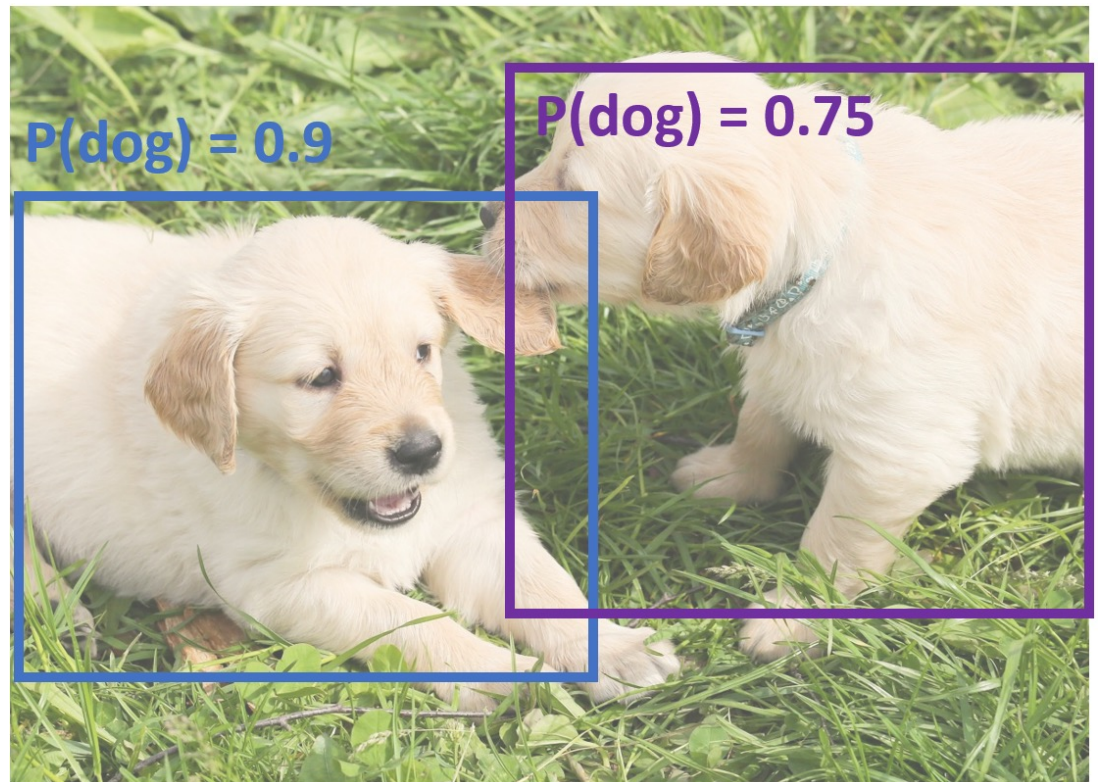
1. Select the highest-scoring box
2. Eliminate lower-scoring boxes with $\text{IoU} > \text{threshold}$
3. If any boxes remain, goto 1

$$\text{IoU}(\blacksquare, \blacksquare) = \mathbf{0.74}$$



Non-Max Suppression

1. Select the highest-scoring box
2. Eliminate lower-scoring boxes with $\text{IoU} > \text{threshold}$
3. If any boxes remain, goto 1



Fast R-CNN

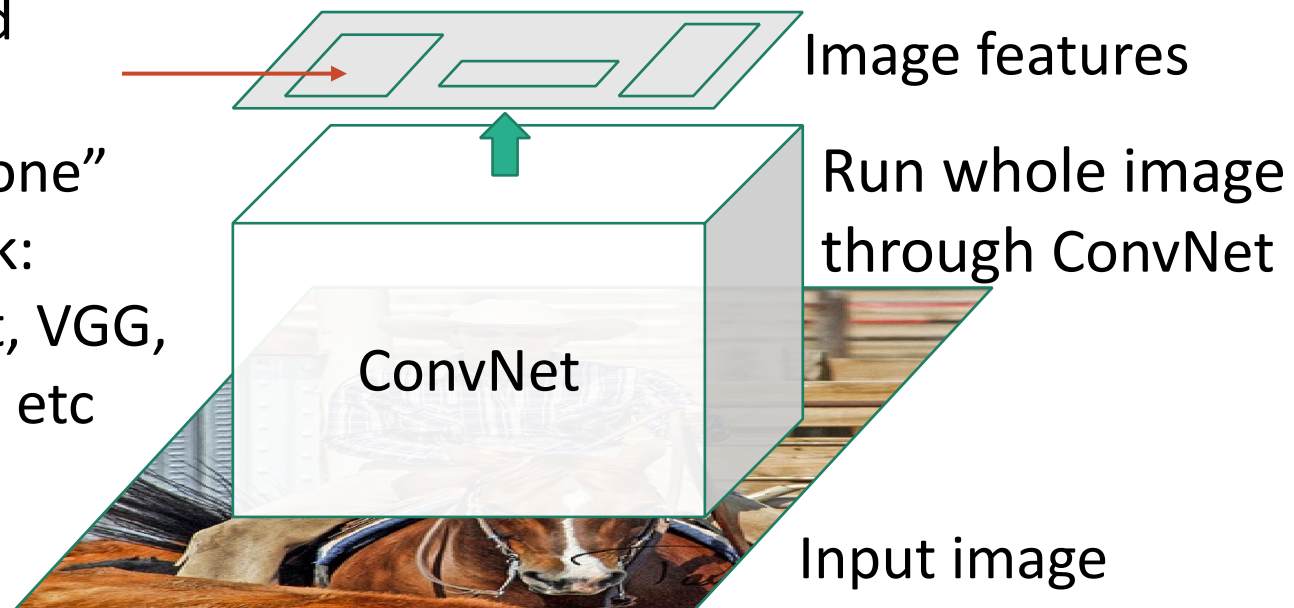


Input image

Fast R-CNN

Regions of Interest (RoIs)
from a proposal
method

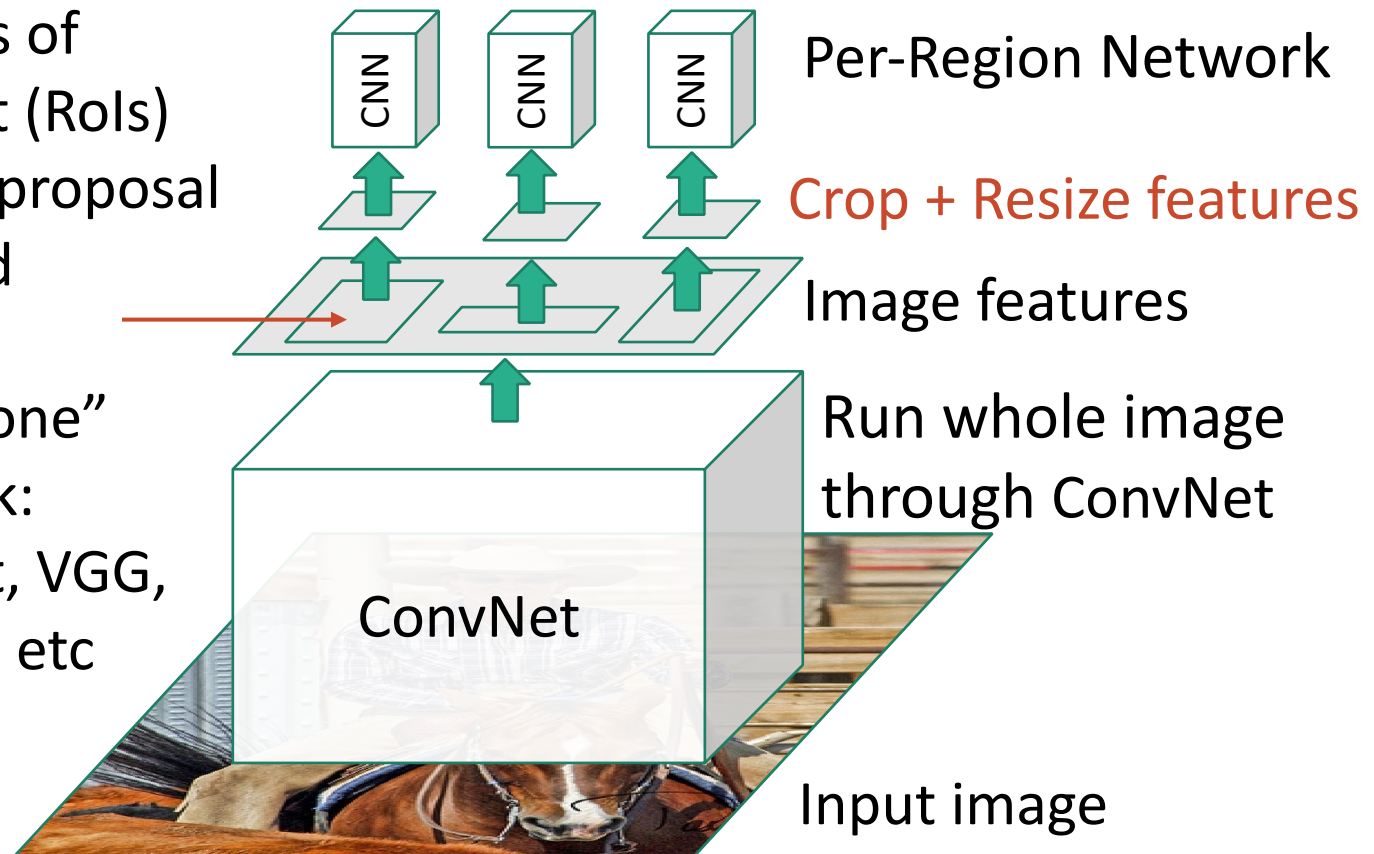
“Backbone”
network:
AlexNet, VGG,
ResNet, etc



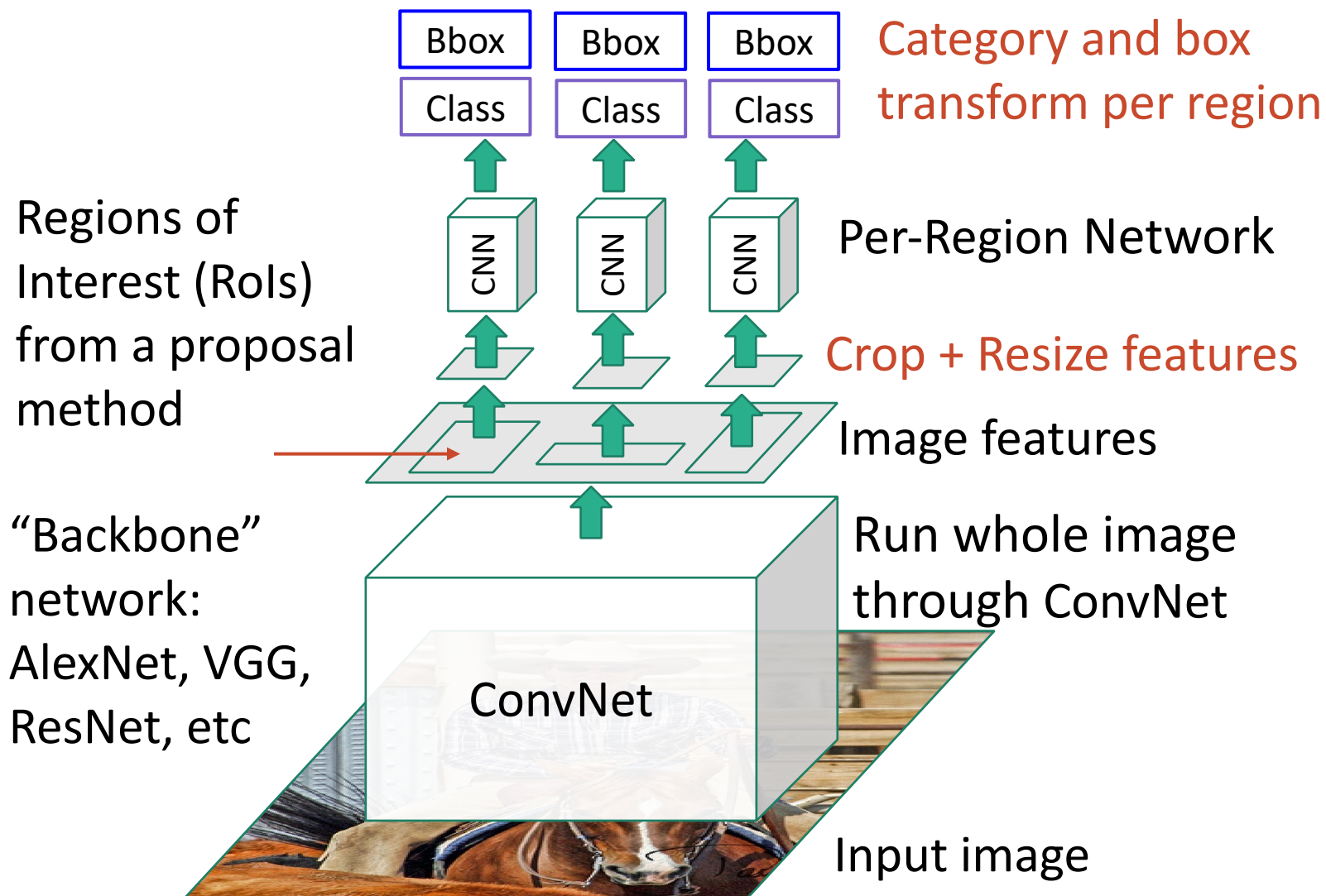
Fast R-CNN

Regions of Interest (RoIs) from a proposal method

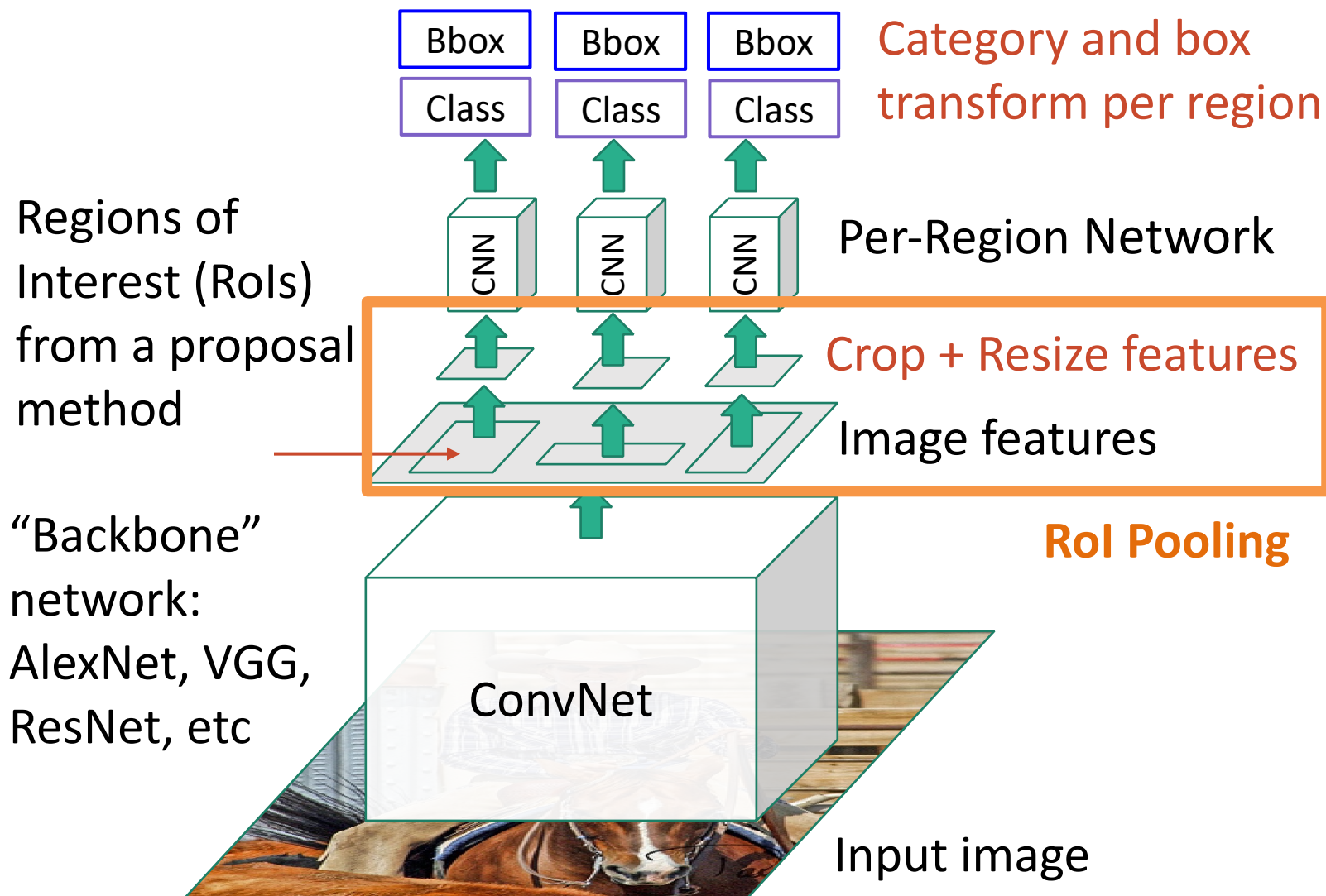
“Backbone” network:
AlexNet, VGG,
ResNet, etc



Fast R-CNN

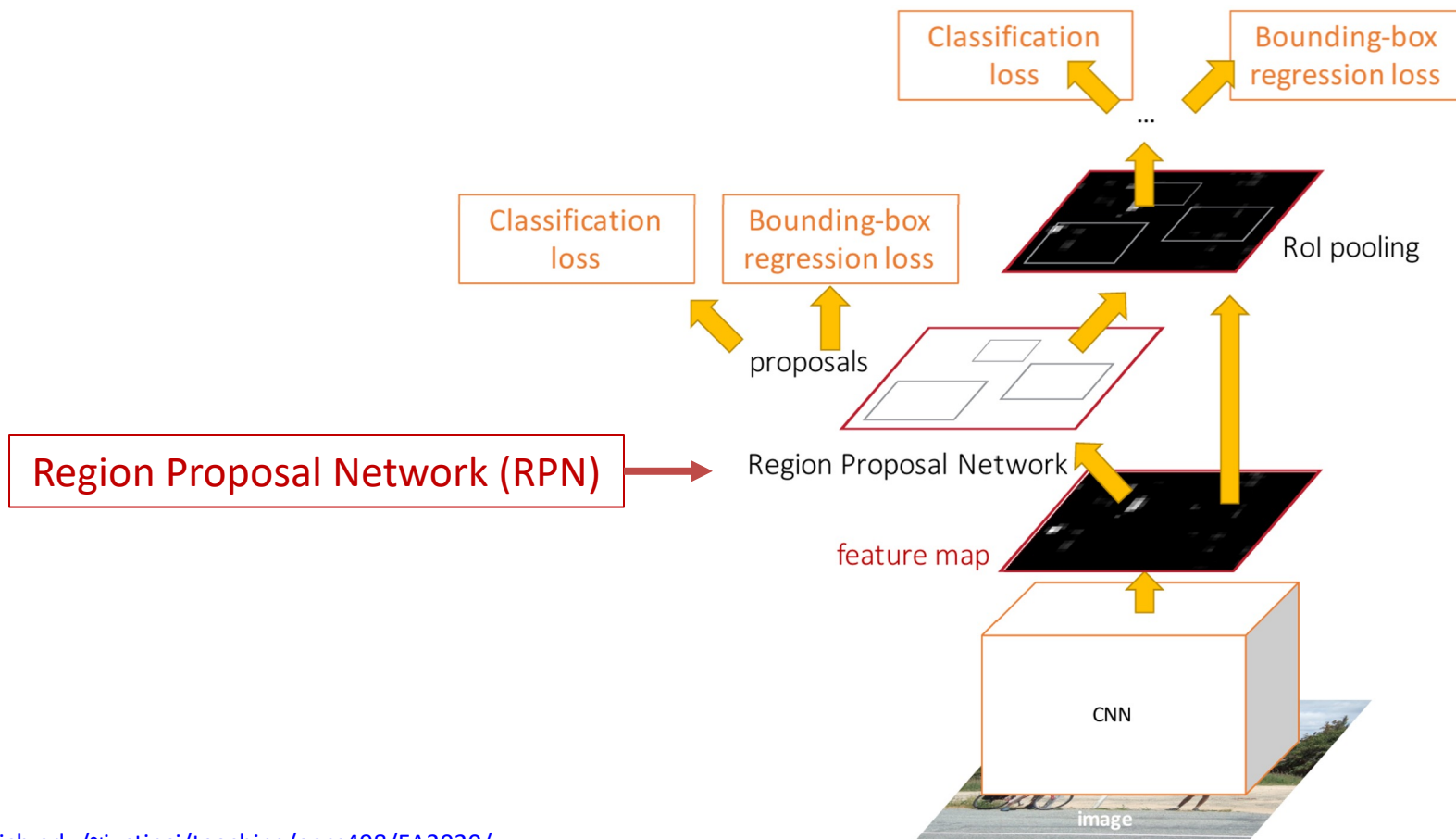


Fast R-CNN



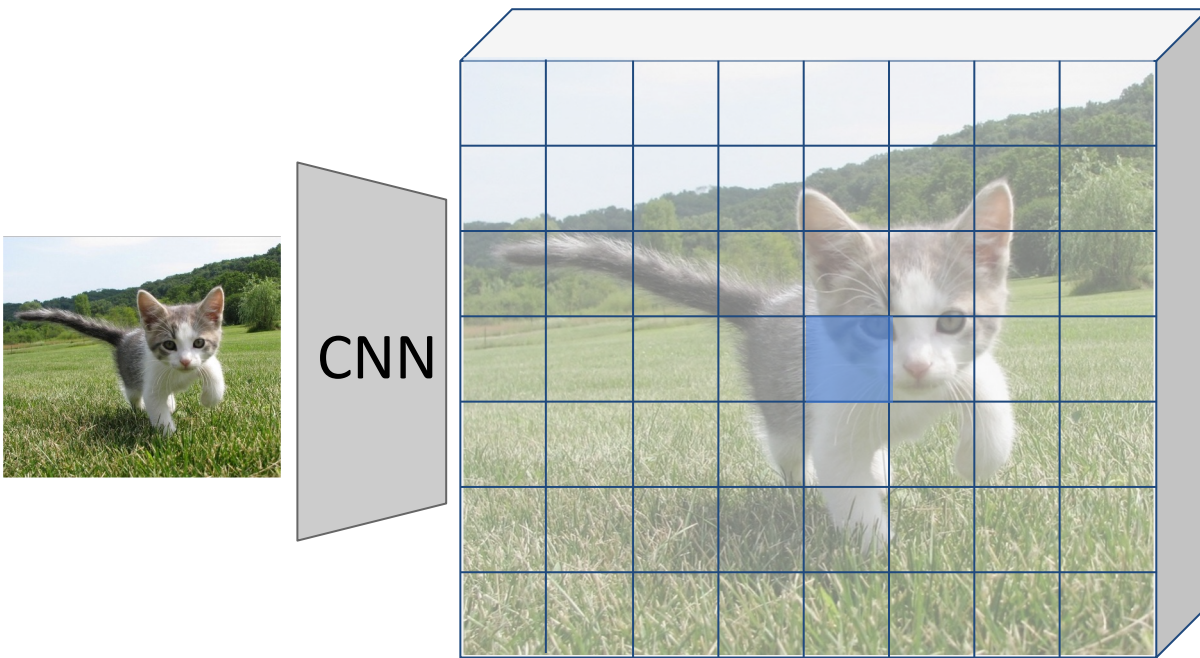
Faster R-CNN

Use CNN to select proposals!



RPN

Imagine an **anchor box** of fixed size at each point in the feature map

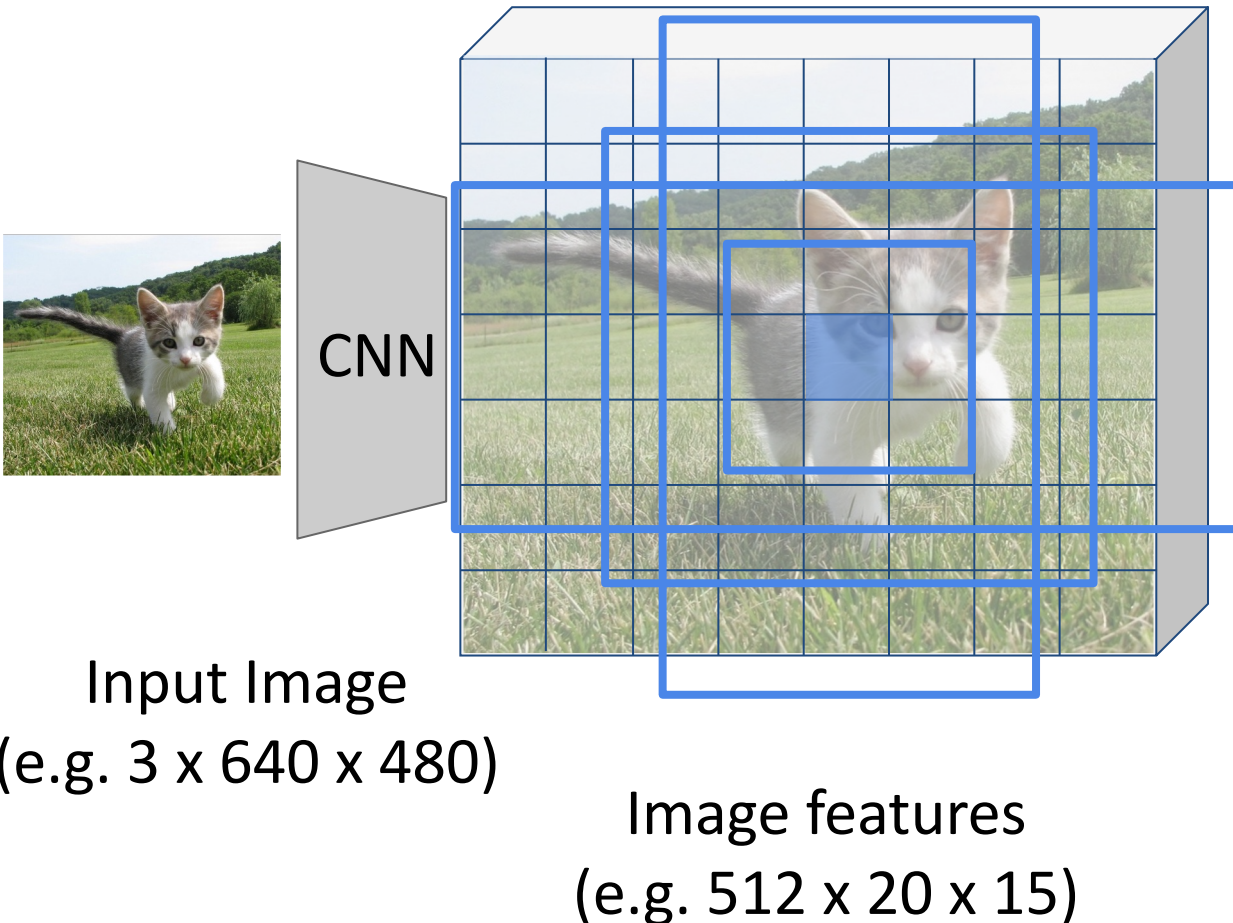


Input Image
(e.g. 3 x 640 x 480)

Image features
(e.g. 512 x 20 x 15)

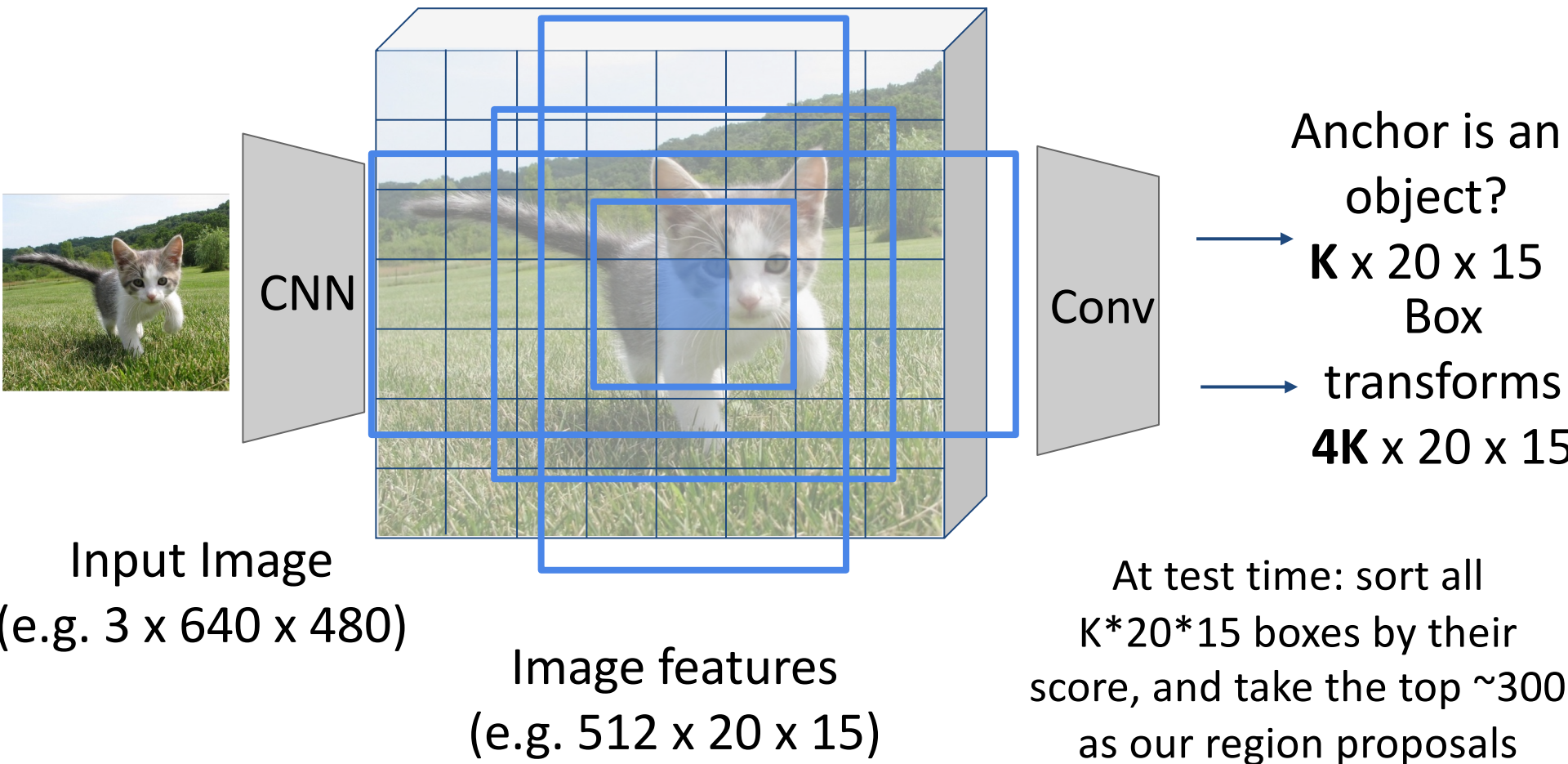
RPN

Imagine an **anchor box** of fixed size at each point in the feature map



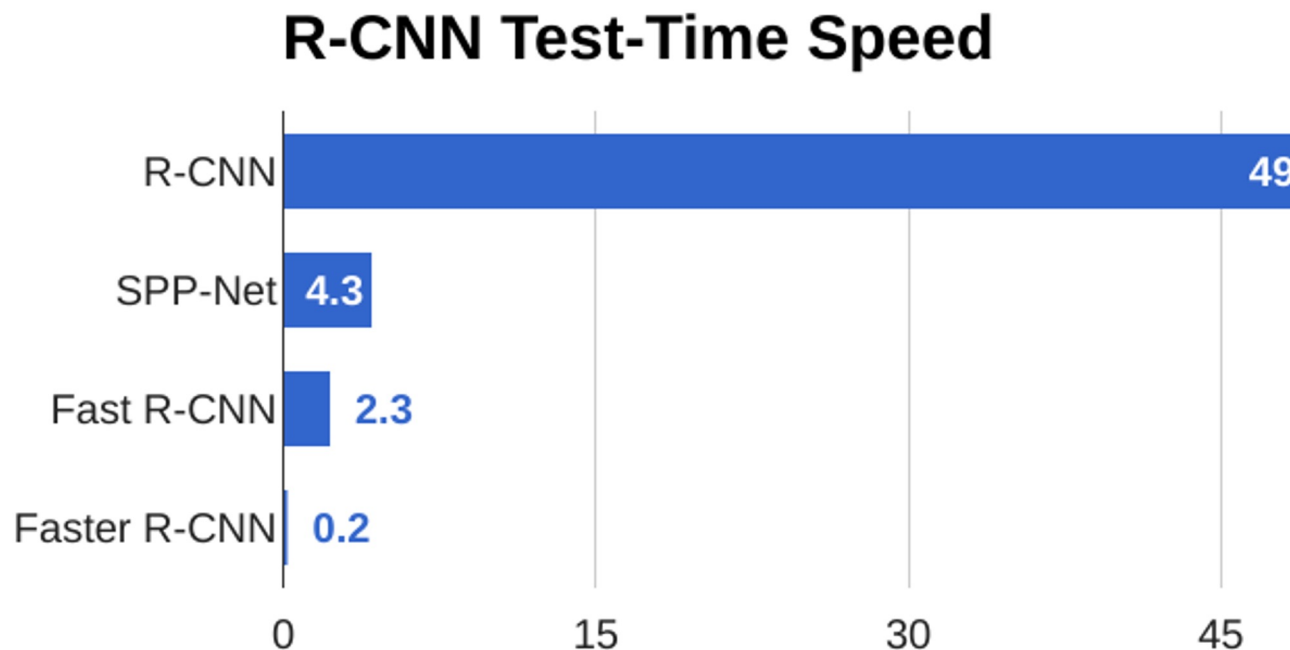
RPN

Imagine an **anchor box** of fixed size at each point in the feature map



Faster R-CNN

Learnable region proposals



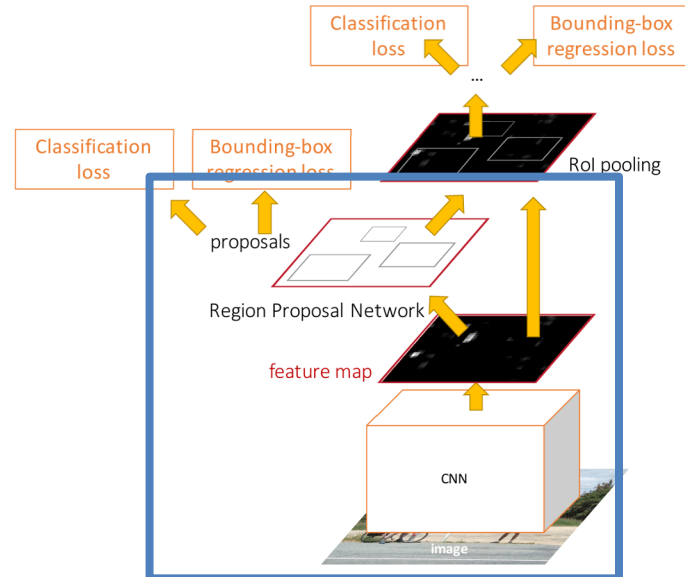
Faster R-CNN

A two-stage object detector

Faster R-CNN

A two-stage object detector

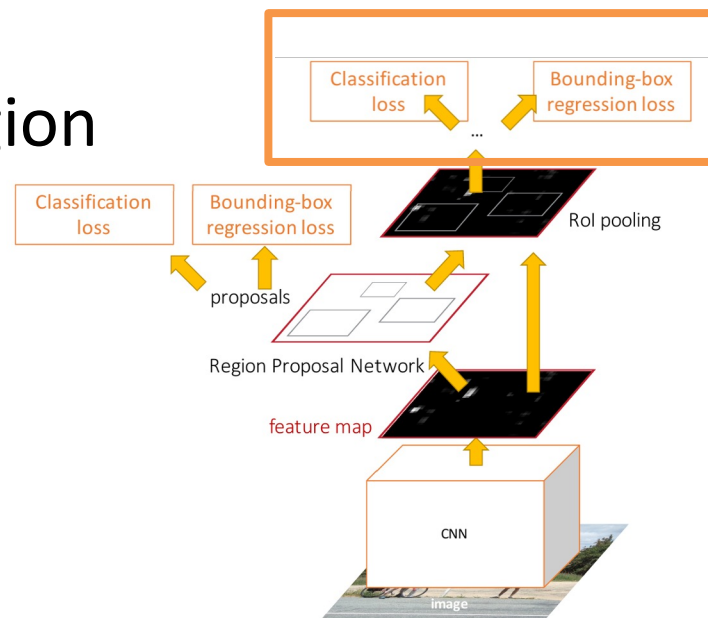
- First stage: run once per image
 - Backbone network
 - RPN



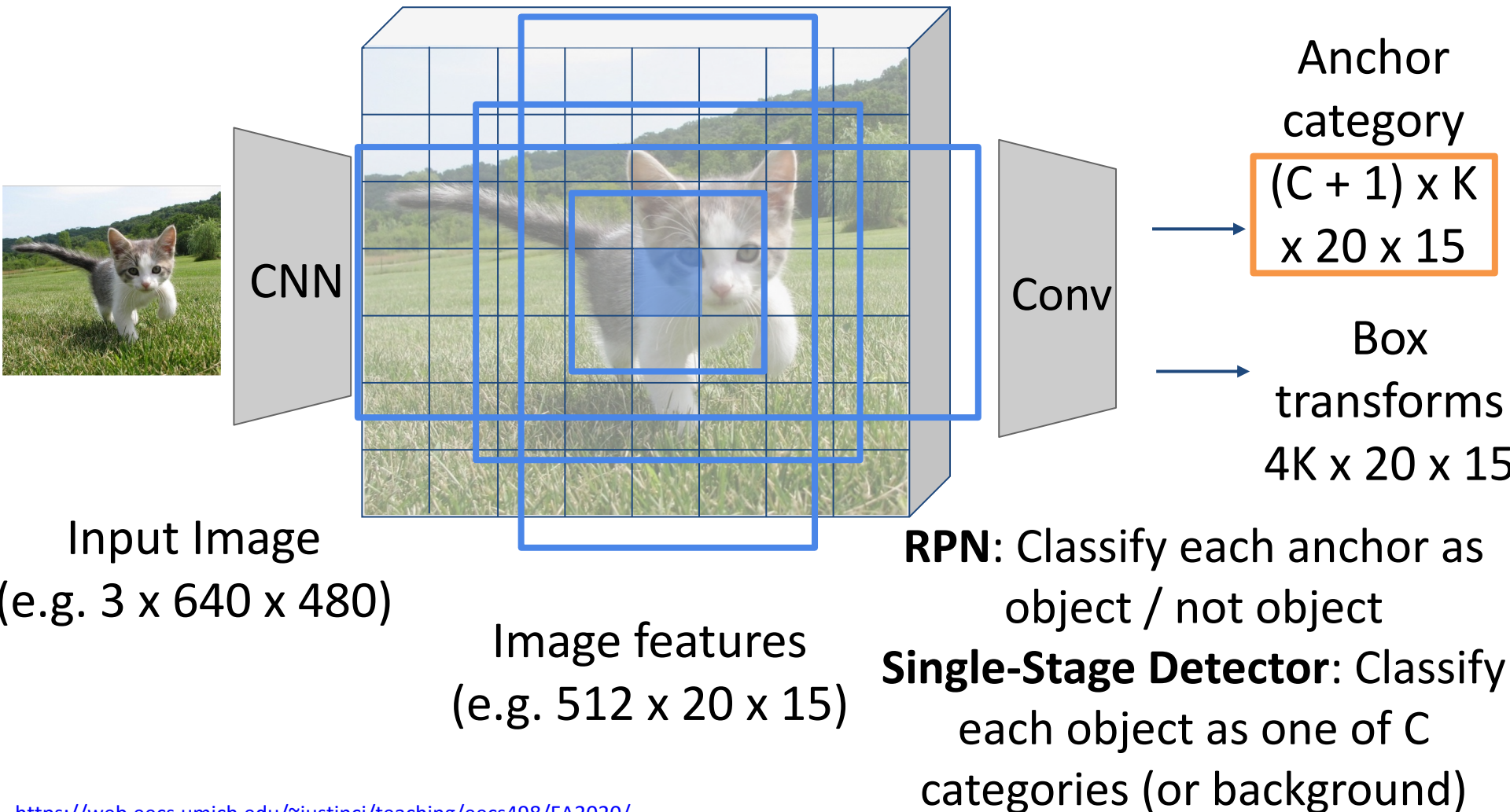
Faster R-CNN

A two-stage object detector

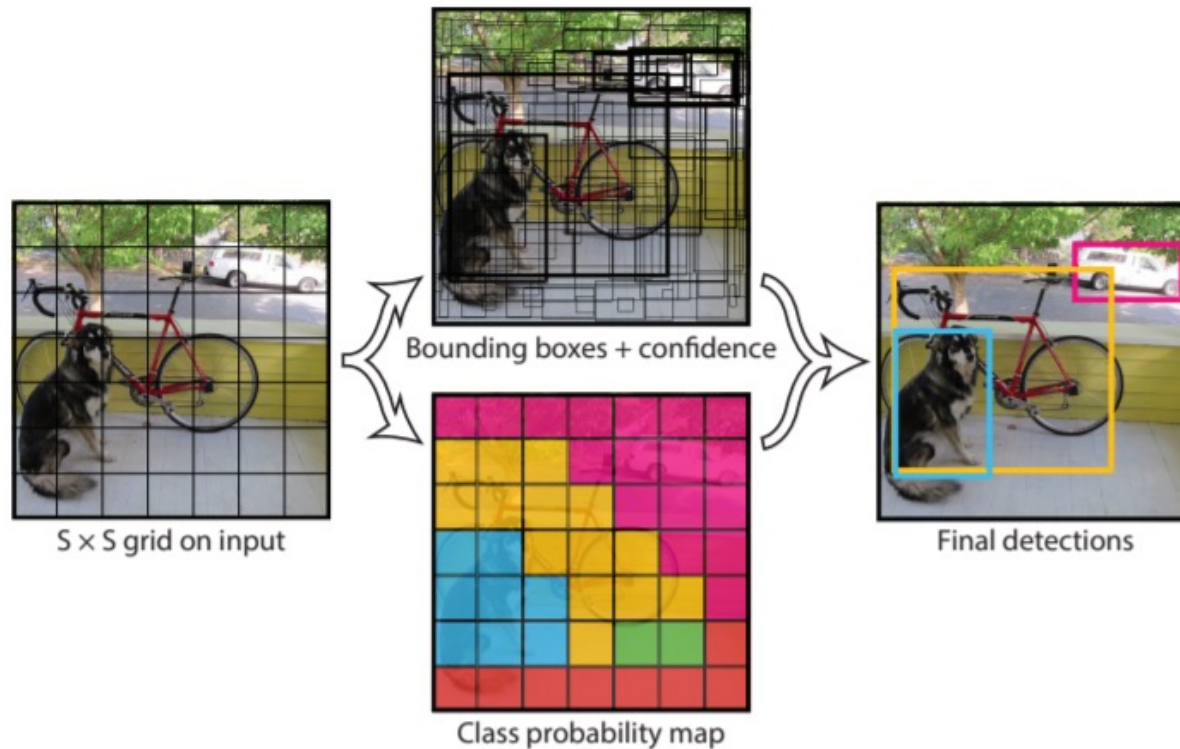
- First stage: run once per image
 - Backbone network
 - RPN
- Second stage: run once per region
 - Crop features: RoI pool / align
 - Predict object class
 - Predict bbox offset



Single-stage object detection



Example: YOLO detector



Two-stage vs. single-stage

Which should I choose?

- Two-stage is generally more accurate
- Single-stage is faster

Today

Semantic segmentation

Object detection

Instance segmentation

Human pose estimation

Optical flow

Instance segmentation

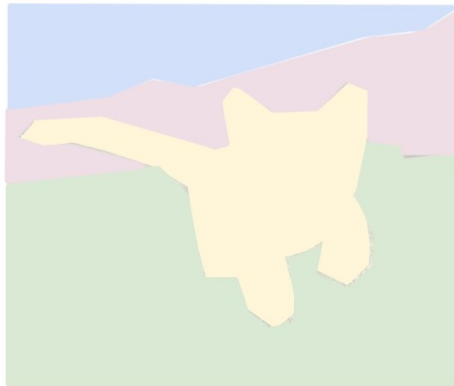
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE,
SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

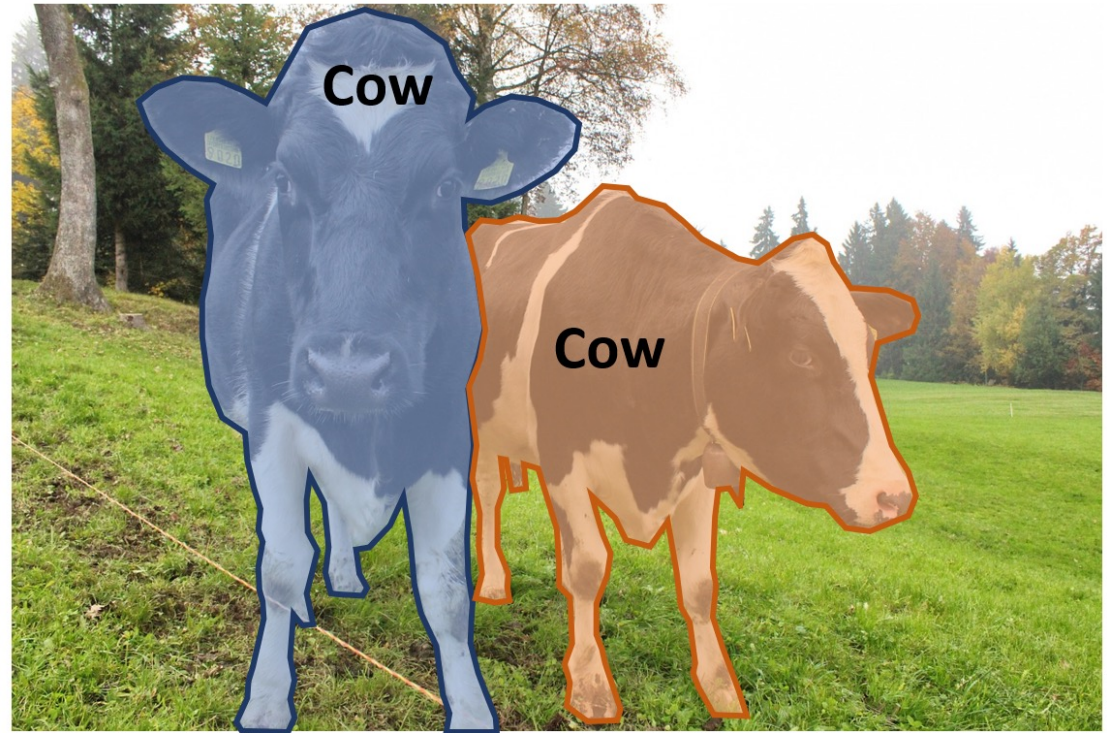
Instance Segmentation



DOG, DOG, CAT

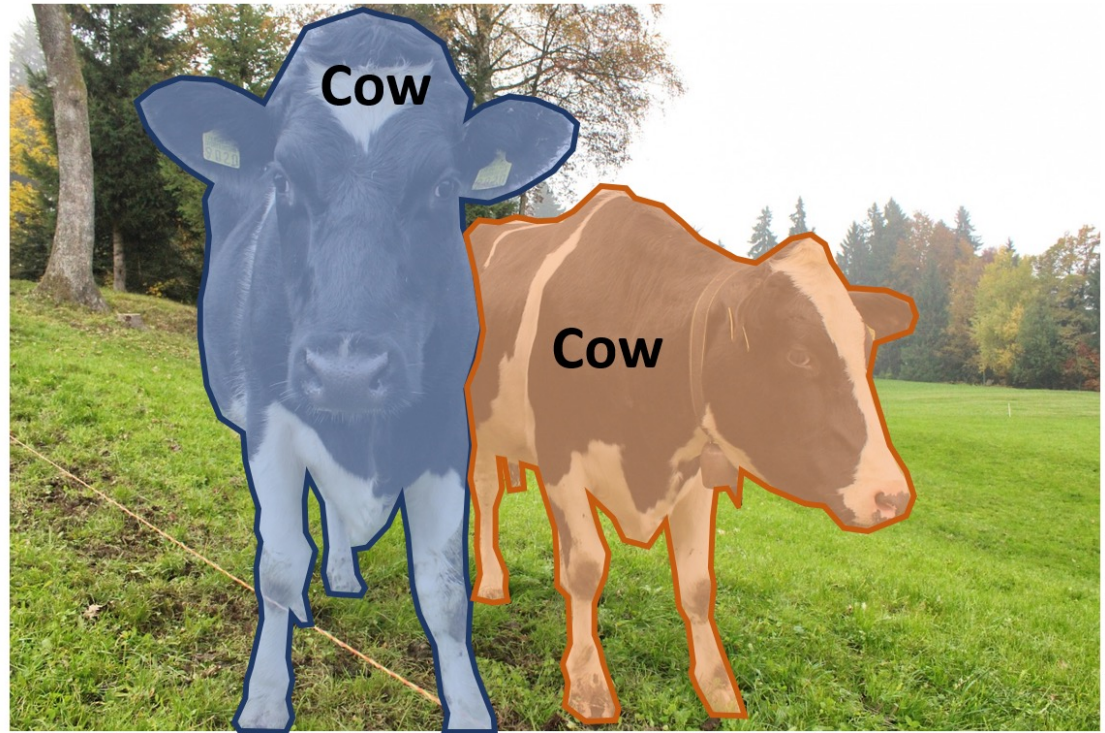
Instance segmentation

Detect all objects in the image, and identify the pixels that belong to each object (Only things)

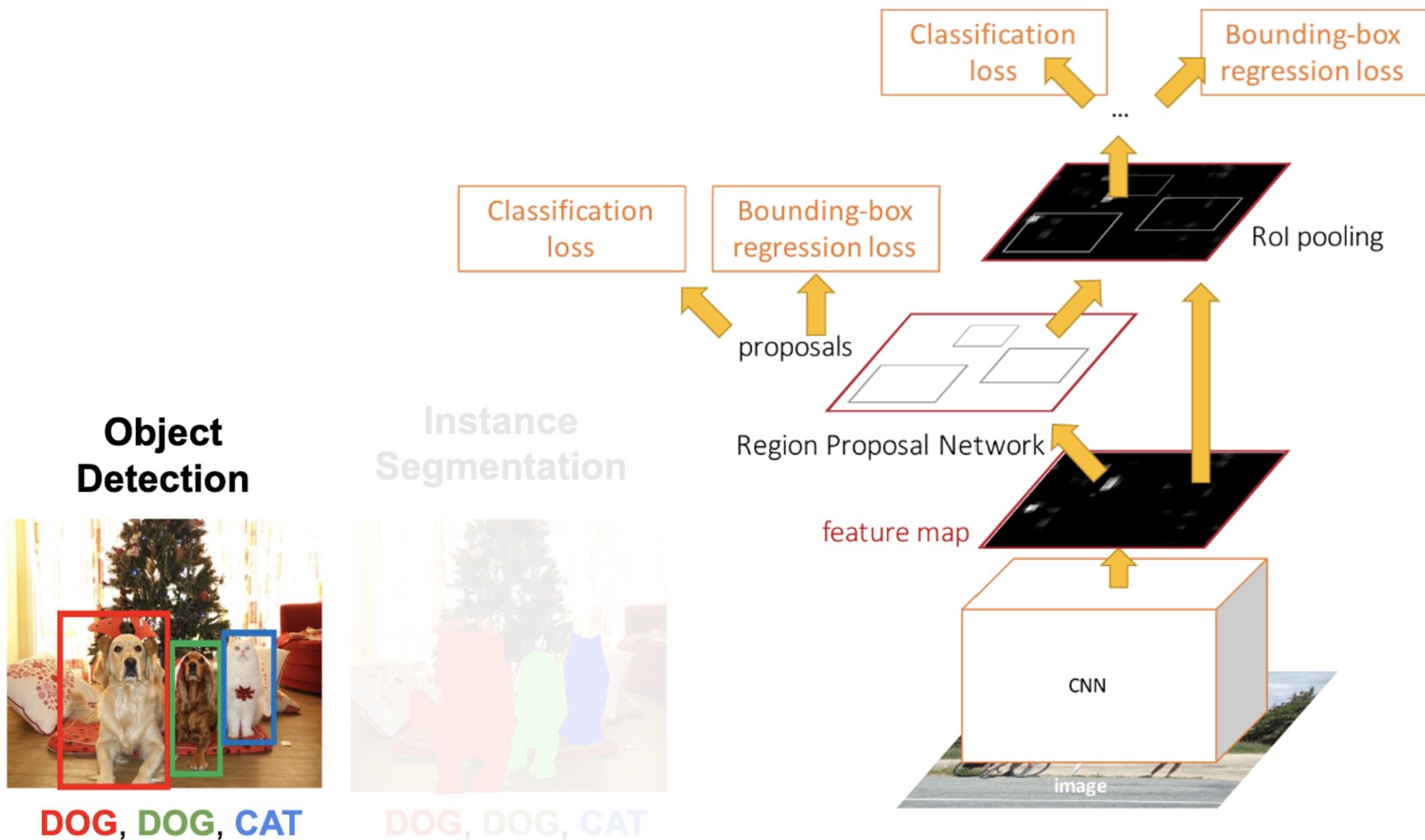


Instance segmentation

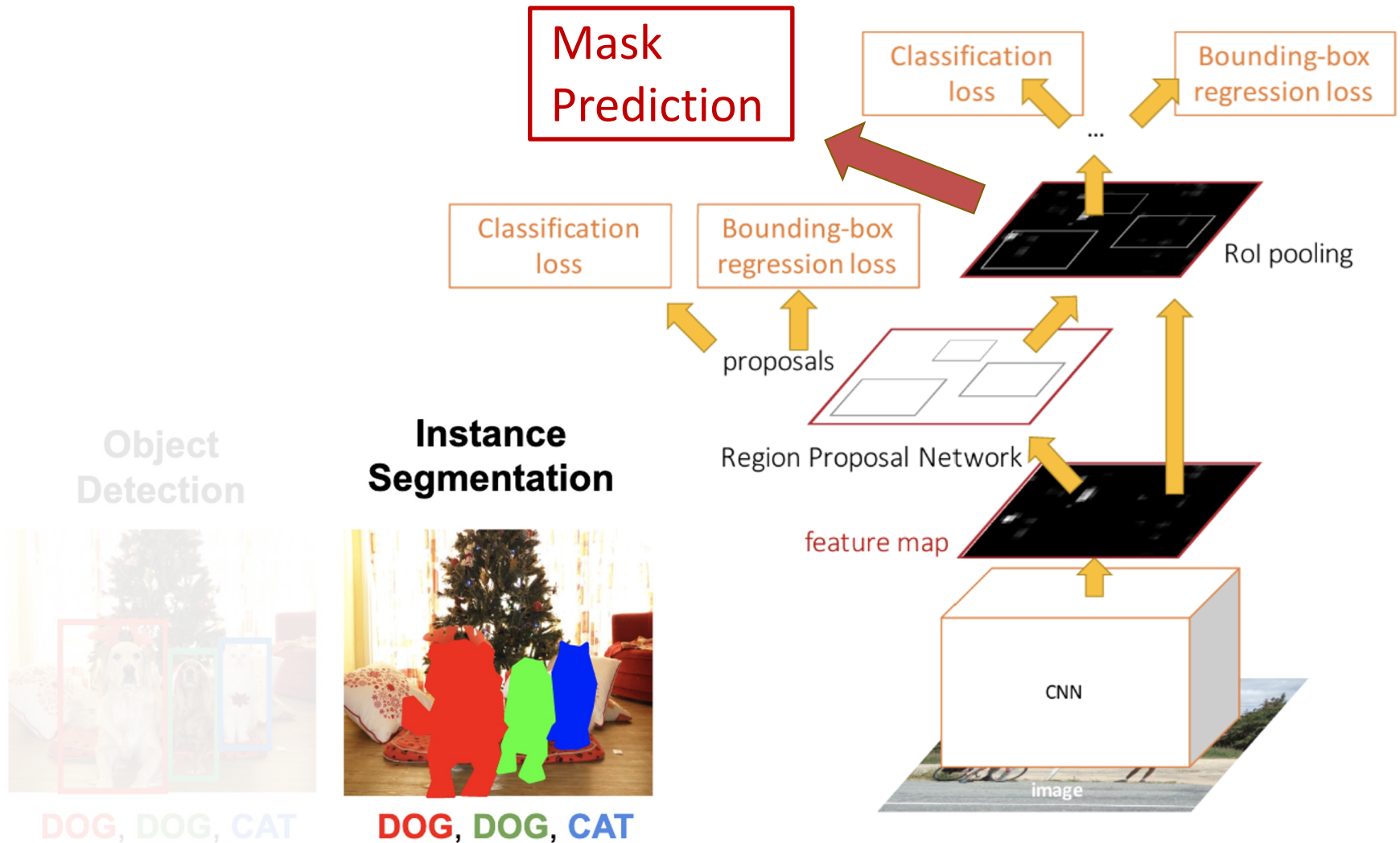
Perform object detection, then predict a segmentation mask for each object



Faster R-CNN

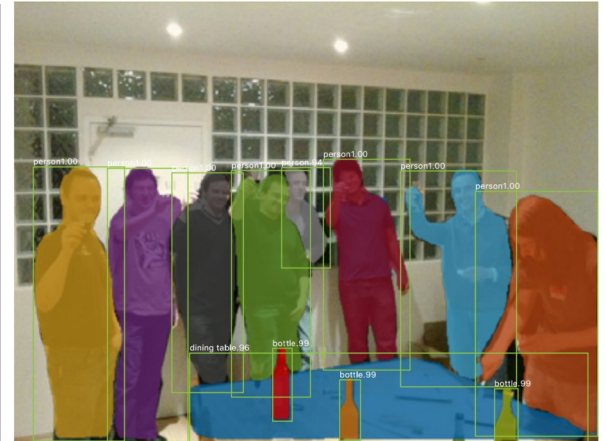
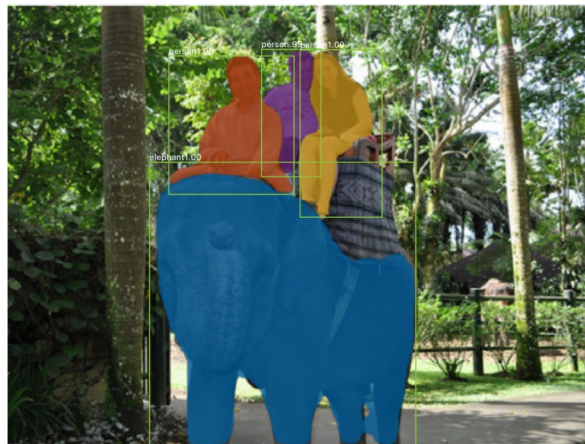


Faster R-CNN + additional head



Mask R-CNN

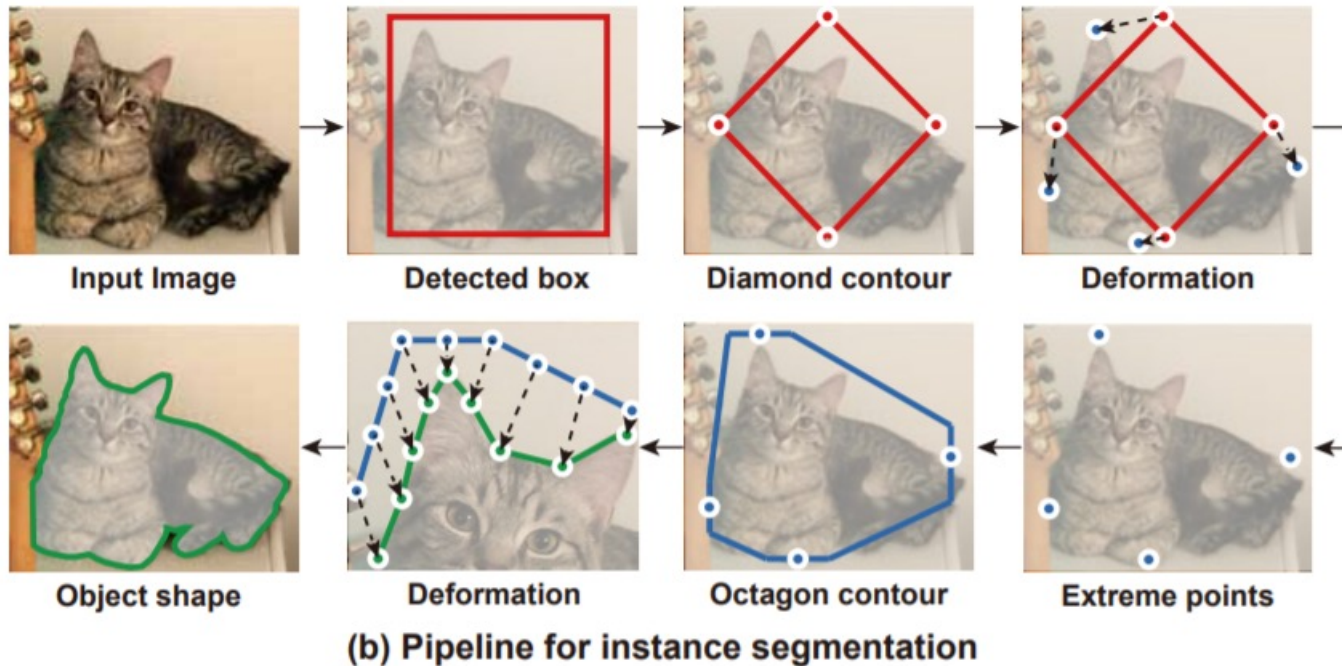
Very good results!



Deep snake

Any other way than object masks?

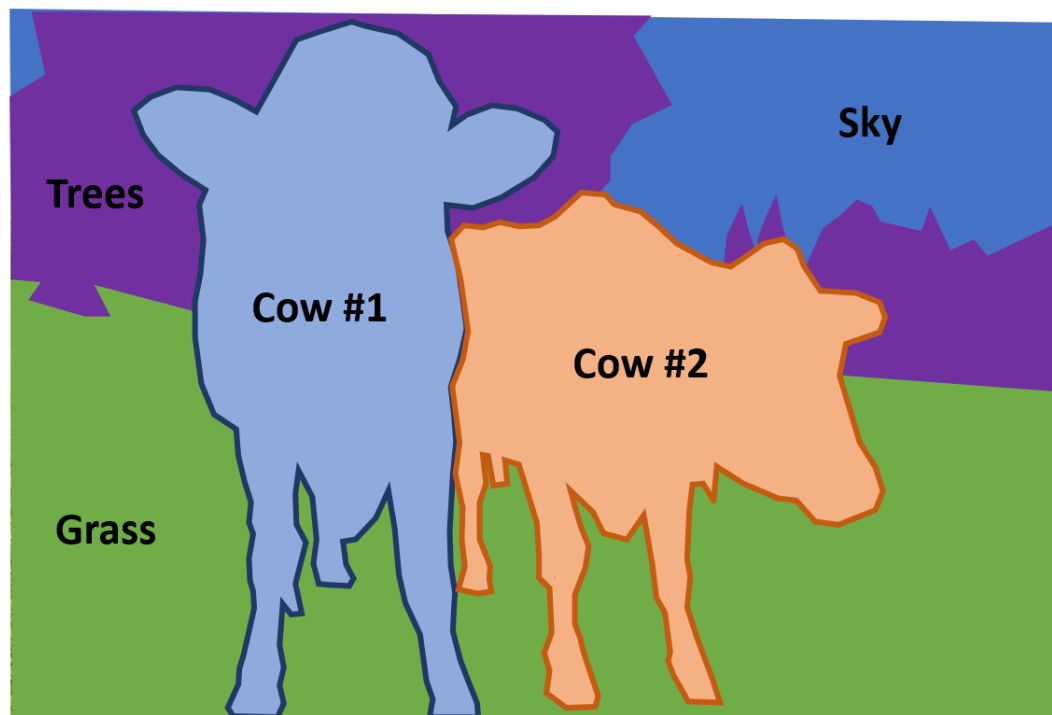
– Contours



Panoptic segmentation

Beyond instance-segmentation

- Label all pixels in the image (both things and stuff)
- For “thing” categories also separate into instances



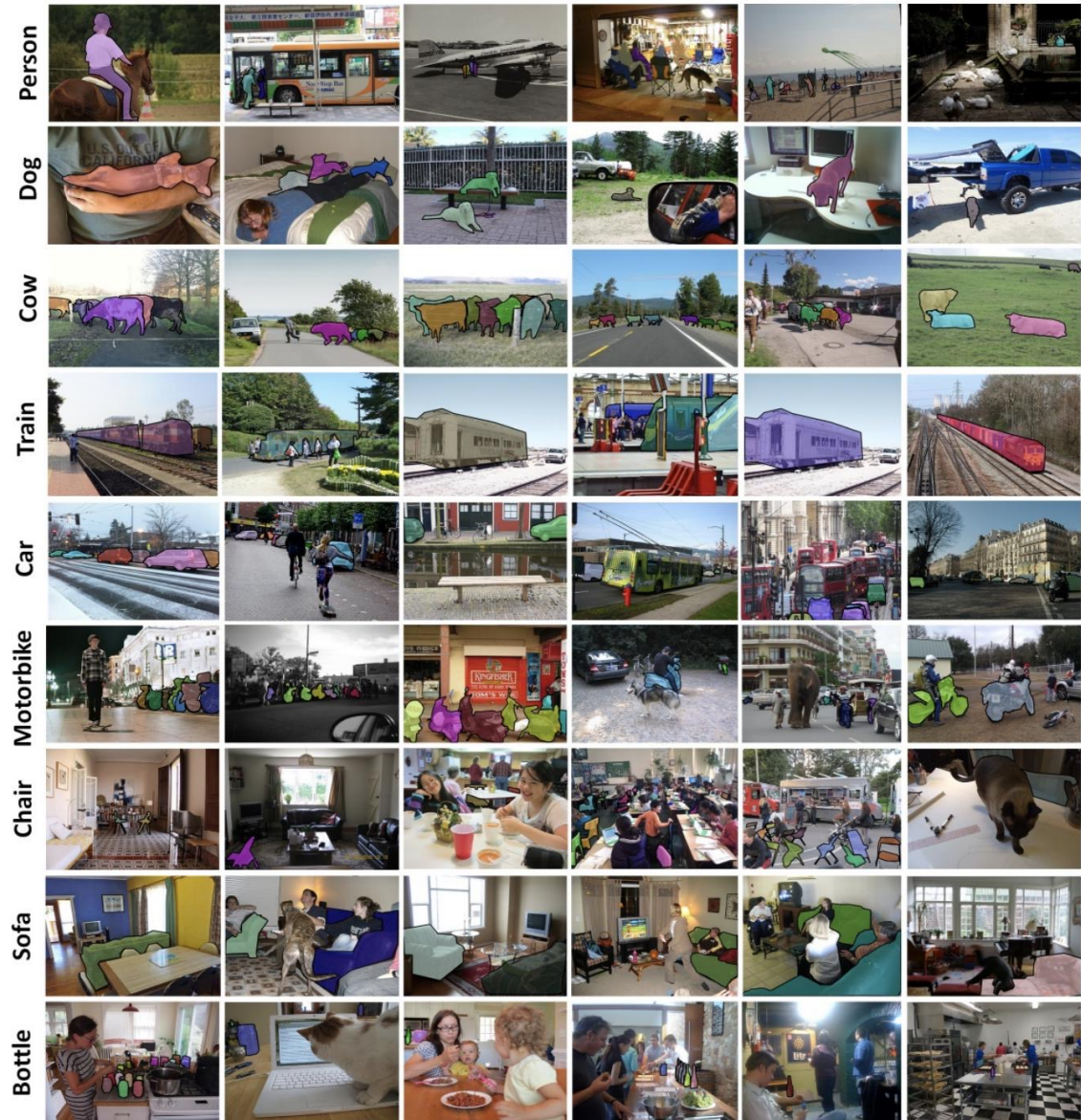
Panoptic segmentation



Datasets

Microsoft COCO

- 118K train
- 5K val
- Contains
 - Object detection
 - Keypoints
 - Instance seg
 - Panoptic seg



Today

Semantic segmentation

Object detection

Instance segmentation

Human pose estimation

Optical flow

Human pose estimation

Represent the pose of a human by locating a set of **keypoints**

e.g. 17 keypoints:

- Nose
- Left / Right eye
- Left / Right ear
- Left / Right shoulder
- Left / Right elbow
- Left / Right wrist
- Left / Right hip
- Left / Right knee
- Left / Right ankle



Human pose estimation

Represent the pose of a human by locating a set of **keypoints**

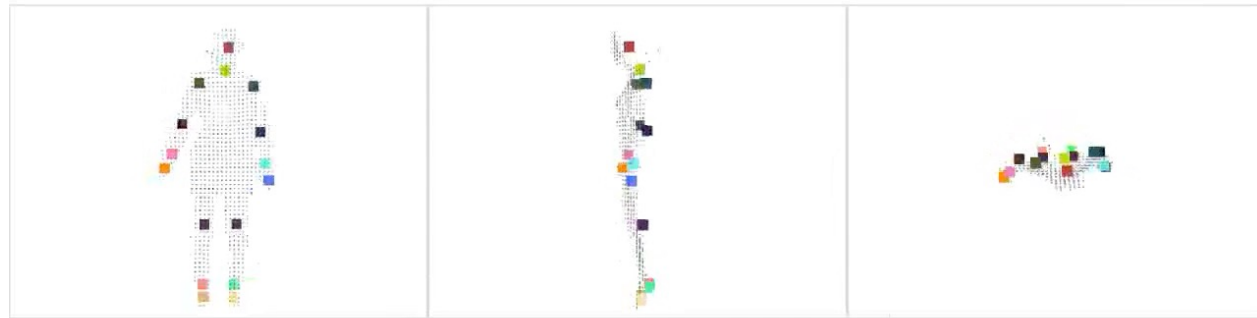
e.g. 17 keypoints:

- Nose
- Left / Right eye
- Left / Right ear
- Left / Right shoulder
- Left / Right elbow
- Left / Right wrist
- Left / Right hip
- Left / Right knee
- Left / Right ankle



Human pose estimation with depth sensors

Microsoft Kinect



Real-time human pose recognition in parts from single depth images. CVPR 2011.

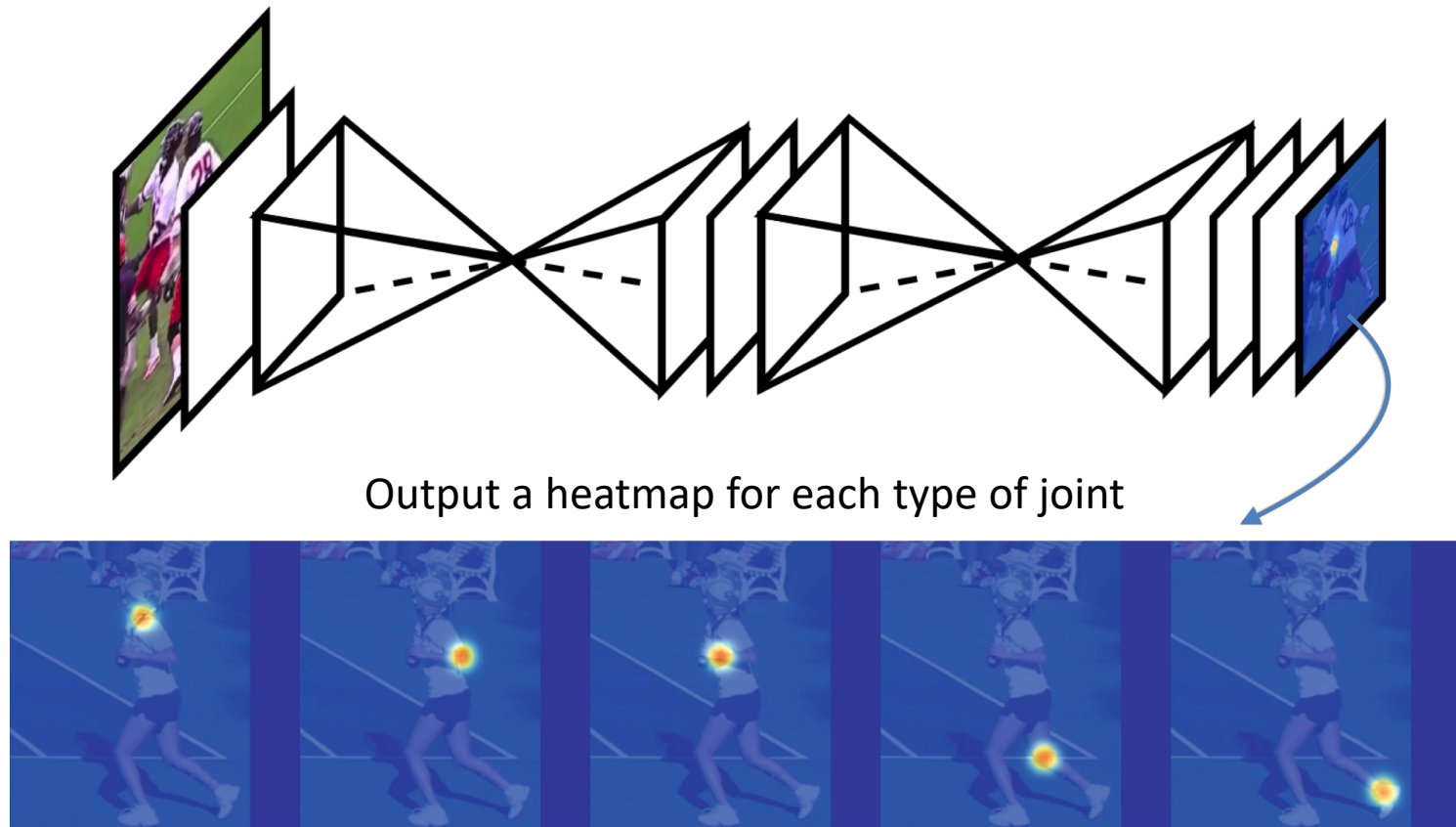
Single human

Directly predict joint locations (a 17×2 vector)



Single human

Represent joint location as the heatmap



Multiple humans

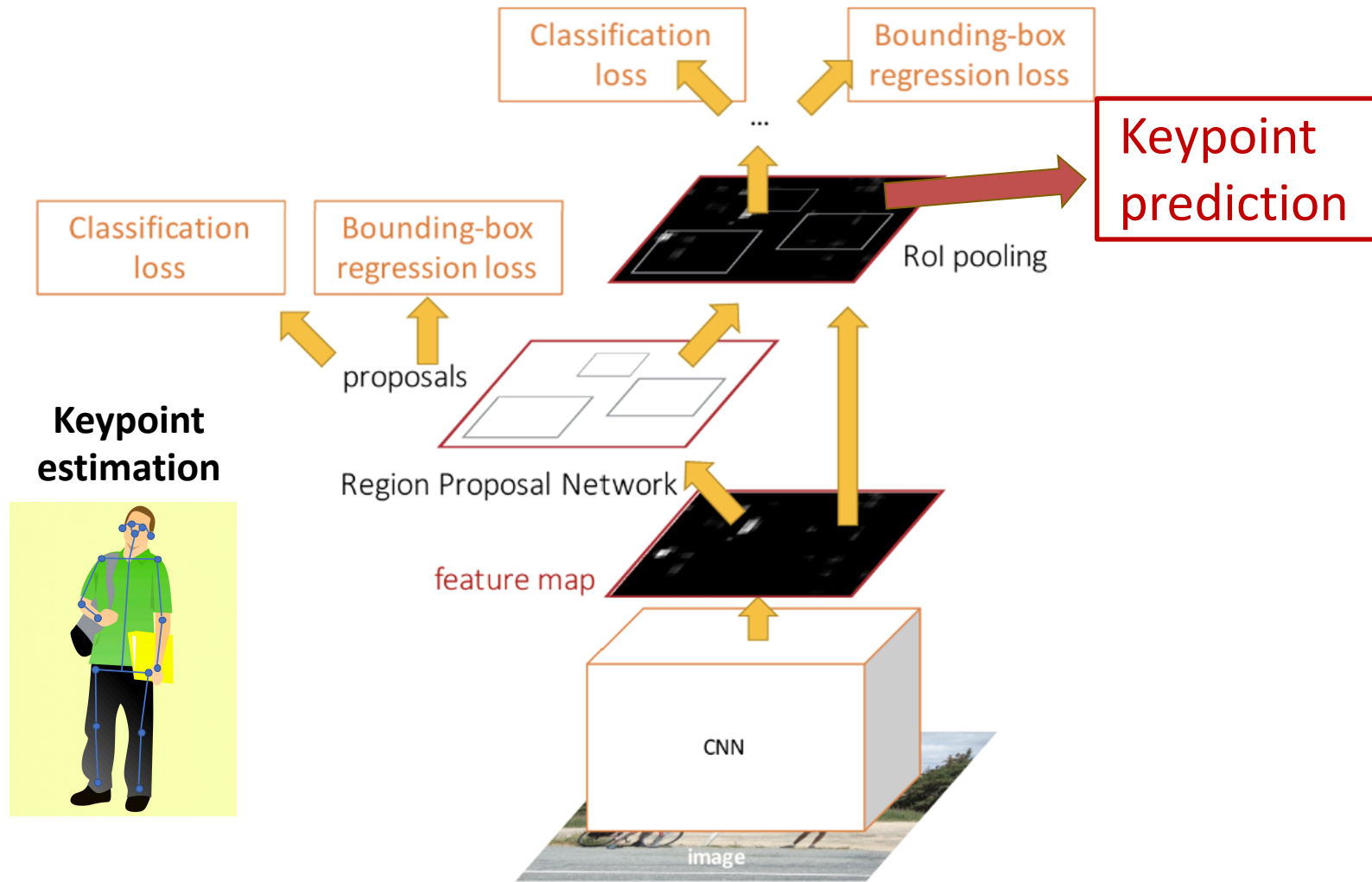
Top-down:

- Detect humans and detect keypoints in each bbox
- Example: Mask R-CNN

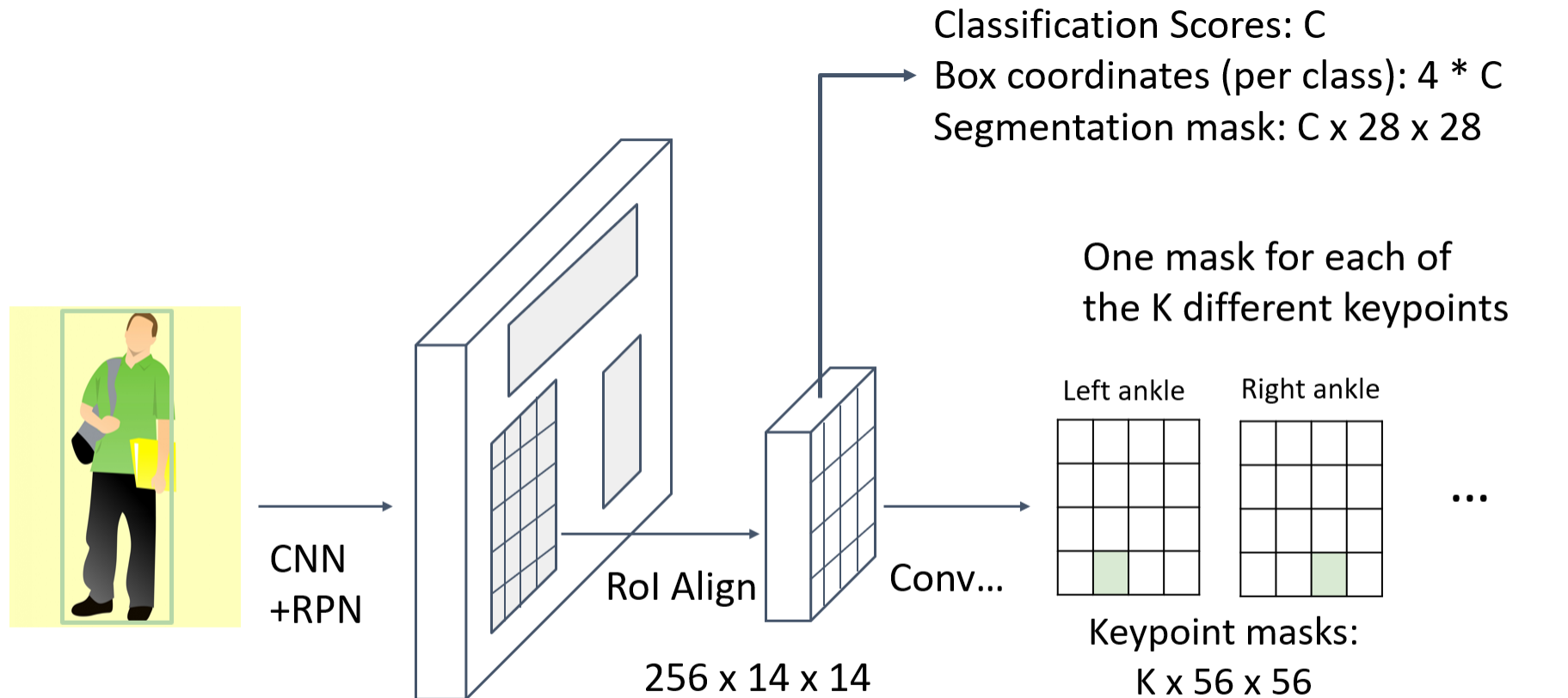
Bottom-up:

- Detect keypoints and group keypoints to form humans
- Example: OpenPose

Mask R-CNN



Mask R-CNN



Ground-truth has one “pixel” turned on per keypoint. Train with softmax loss

Mask R-CNN



OpenPose

Part affinity fields



(a) Input Image



(b) Part Confidence Maps



(c) Part Affinity Fields

OpenPose

Link parts based on part affinity fields

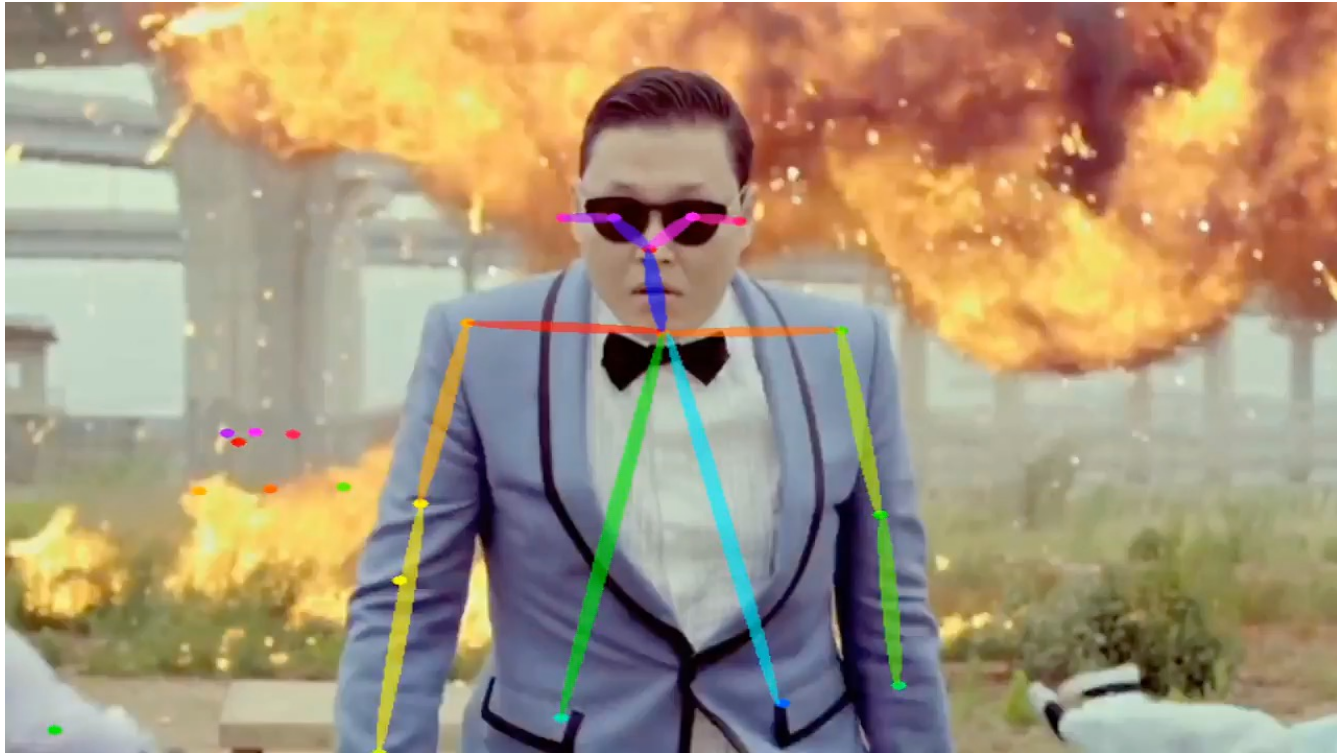


(d) Bipartite Matching



(e) Parsing Results

OpenPose



https://youtu.be/mxKIUO_tjcg

Top-down or bottom-up?

Which should I choose?

- Top-down is generally more accurate
- Bottom-up is faster

Today

Semantic segmentation

Object detection

Instance segmentation

Human pose estimation

Other tasks

Video classification



Swimming
Running
Jumping
Eating
Standing

Input video:
 $T \times 3 \times H \times W$

Video classification



Images: Recognize **objects**



Dog
Cat
Fish
Truck



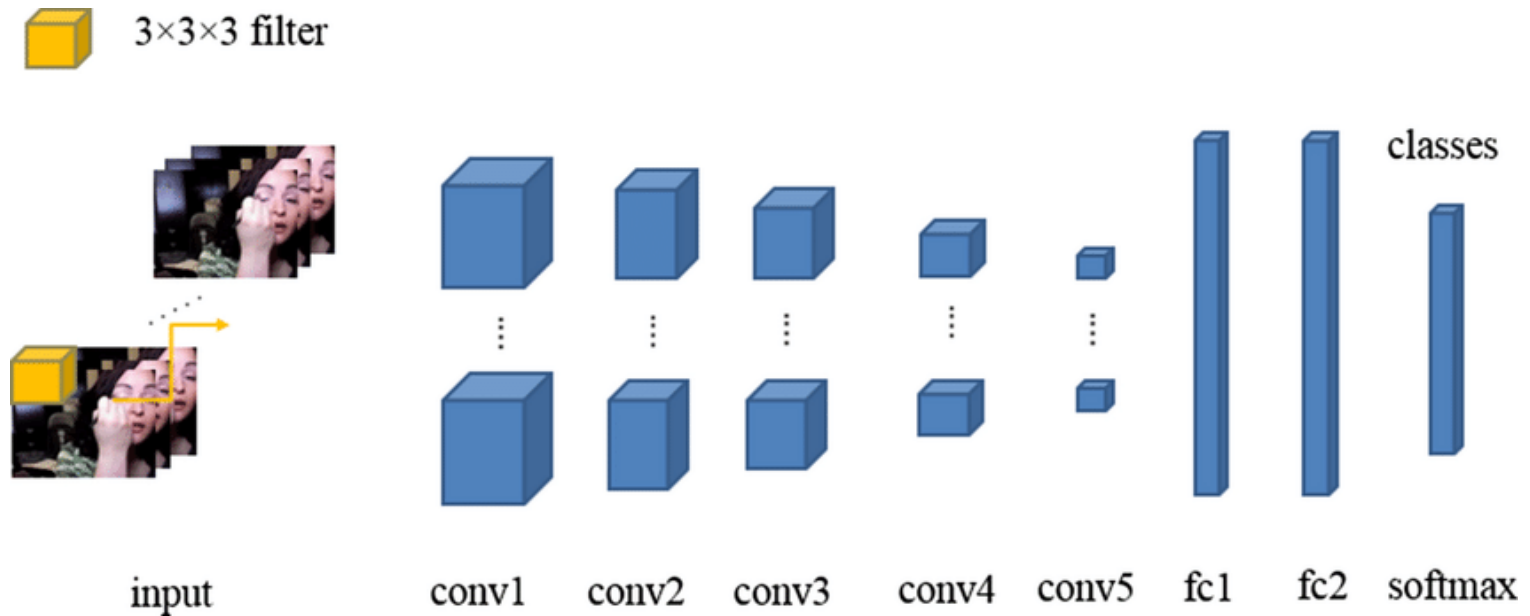
Videos: Recognize **actions**



Swimming
Running
Jumping
Eating
Standing

Video classification

Use 3D CNN



Temporal action localization

Given a long untrimmed video sequence, identify frames corresponding to different actions

Running

Jumping



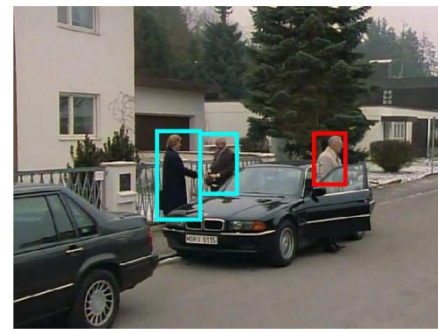
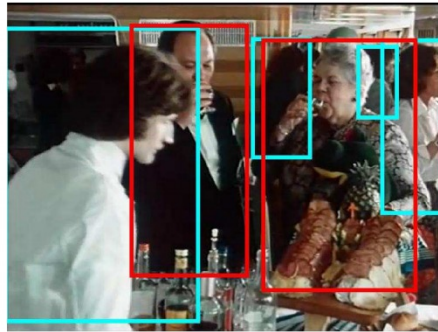
– Generate proposals then classify

Spatial-temporal detection

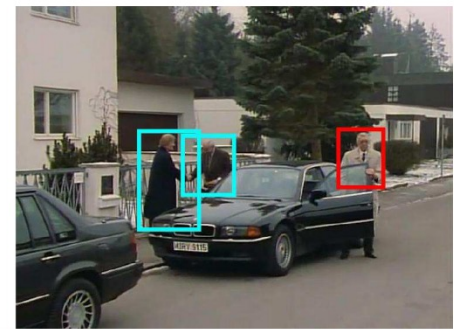
Given a long untrimmed video, detect all the people in space and time and classify the activities they are performing



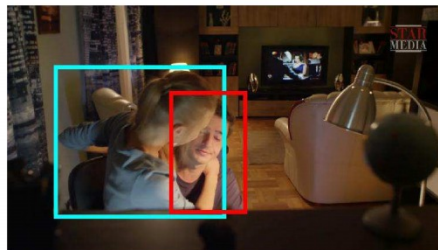
clink glass → drink



open → close



grab (a person) → hug



look at phone → answer phone



Multi-object tracking

Identify and track objects belonging to one or more categories without any prior knowledge about the appearance and number of targets.



Papers with code

Where can I find state-of-the-art?

Browse State-of-the-Art

5,826 benchmarks 2,576 tasks 60,121 papers with code

paperswithcode.com

Computer Vision



Semantic
Segmentation

📊 202 benchmarks
2256 papers with code



Image
Classification

📊 274 benchmarks
1946 papers with code



Object
Detection

📊 262 benchmarks
1705 papers with code



Image
Generation

📊 168 benchmarks
755 papers with code



Denoising

📊 100 benchmarks
720 papers with code

► [See all 1166 tasks](#)

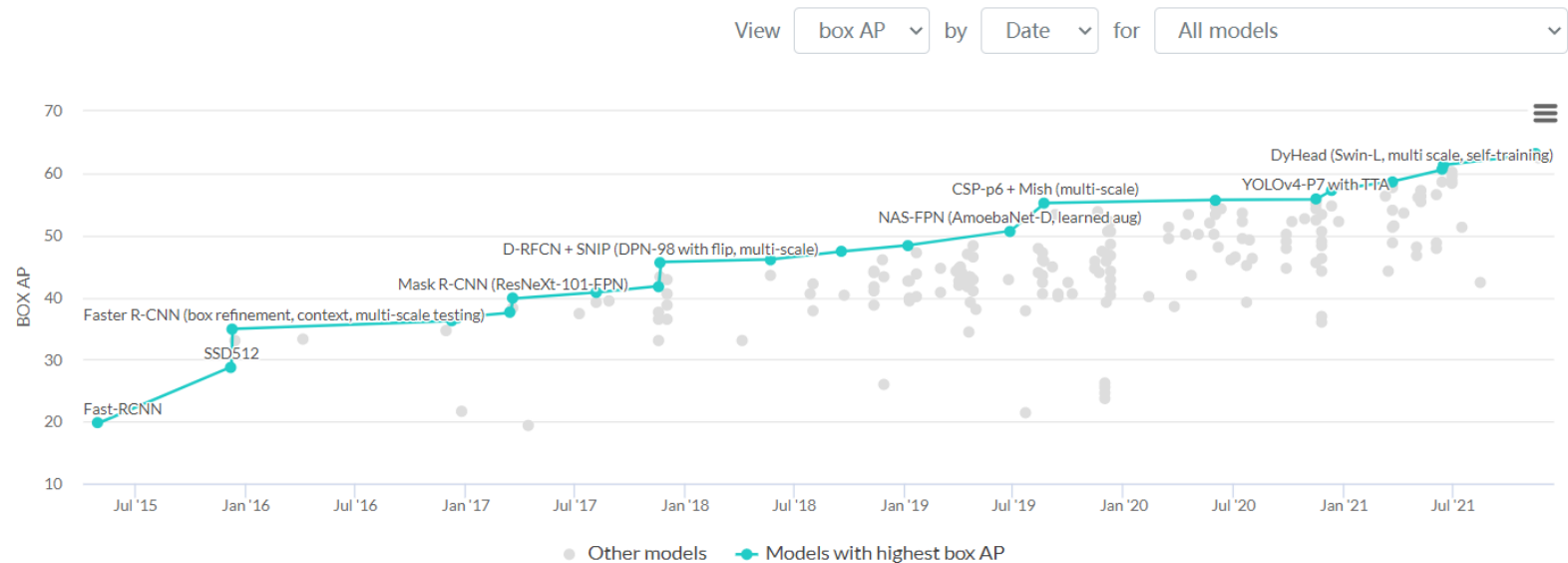
Papers with code

Choose a task.

Object Detection on COCO test-dev

Leaderboard

Dataset



Papers with code

Take a look at the leaderboard.

- Provided with the links to paper and code.

Rank	Model	 AP	AP50	AP75	APs	APM	APL	Extra Training Data	Paper	Code	Result	Year	Tags 
1	SwinV2-G (HTC++)	63.1						✓	Swin Transformer V2: Scaling Up Capacity and Resolution			2021	swin-transformer
2	Florence-CoSwin-H	62.4						×	Florence: A New Foundation Model for Computer Vision			2021	
3	Soft Teacher + Swin-L (HTC++, multi-scale)	61.3						✓	End-to-End Semi-Supervised Object Detection with Soft Teacher			2021	multiscale Swin-Transformer

Questions?